

Project initial prompt-1

I am building an AI Voice Calling Agent as a serious project and I want you to act as a **senior voice-AI architect, telecom engineer, AI systems engineer and product strategist**.

Do not assume that the technologies or providers I mention are the best. I want an **independent second opinion** based on current information available on the internet. Research the current ecosystem where necessary and challenge weak assumptions.

PROJECT

We are building a highly natural, real-time AI voice calling platform.

The initial version will focus **only on English**. Once the English voice experience reaches a very high level of quality, we will expand it to **Telugu**, and eventually support multiple Indian and international languages.

The goal is not simply to make an AI that can place a phone call.

The goal is to build a voice agent that feels as close to a natural human phone conversation as technically possible.

The agent should be able to:

- Make outbound phone calls.
- Conduct natural conversations.
- Listen continuously and understand the caller's response.
- Handle interruptions naturally.
- Stop speaking when the person interrupts.
- Detect when the person has finished speaking.
- Handle pauses, hesitation and incomplete sentences.
- Maintain conversation context.
- Respond quickly enough that the conversation does not feel robotic.
- Collect structured information from the conversation.
- Determine the outcome of the call.
- Generate a transcript.
- Generate a structured summary/result.
- Track the status of every call.
- Handle multiple calls concurrently.
- Maintain an independent conversation state for every active call.

The system should eventually be **domain-independent** rather than being designed only for sales or lead qualification.

For example, one use case could be a college contacting absent students.

An administrator could upload a list of students containing information such as:

- Name
- Phone number
- Student ID
- Other relevant information

The system would call the students individually and ask why they were absent.

After the campaign, the platform should provide results such as:

- Answered
- Not answered
- Busy
- Failed
- Reason for absence
- Expected return/date
- Other relevant information
- Call duration
- Transcript
- AI-generated summary

The administrator should be able to export the final structured results, for example as an Excel/CSV file.

EXPECTED USER FLOW

Administrator

Website



Upload CSV/Excel containing contacts



Review contacts



Start calling campaign



System initiates calls

↓

AI conducts conversations

↓

Calls are independently tracked

↓

Transcripts and structured information are generated

↓

Administrator views campaign results

↓

Administrator exports results

Authentication and authorization are **not required during the initial development/prototype phase**. They will be added when moving toward production.

TELEPHONY REQUIREMENT

The system needs to make real phone calls using a **proper Indian telephone number**.

We do not want the initial system to depend on an international number because the project will be demonstrated in India and calls should appear legitimate and trustworthy to recipients.

We need to understand the best current options for:

- Indian virtual numbers
- Cloud telephony
- Programmable voice
- SIP
- Voice APIs
- Real-time bidirectional audio
- Outbound calling
- Call status tracking
- Concurrent calls
- Caller ID
- Number provisioning
- KYC requirements
- DLT/regulatory requirements
- Pricing

- Number rental
- Per-minute calling costs
- Concurrency limits
- API/WebSocket support
- Academic/student access if available

Do not assume that a normal SIM/eSIM is the correct architecture. Evaluate **SIM/eSIM, virtual numbers, cloud telephony, SIP providers and other approaches** and explain which is actually appropriate for this project.

MOST IMPORTANT REQUIREMENT: VOICE QUALITY

The primary differentiator of this project should be **conversation quality**, not merely the ability to make calls.

The agent should not feel like:

"User speaks → AI waits → AI thinks → AI speaks."

We want a natural conversation with:

- Very low perceived latency
- Natural turn-taking
- Natural pauses
- Fast response generation
- Reliable interruption handling
- Barge-in support
- Accurate speech recognition
- Natural speech synthesis
- Appropriate pacing
- Natural sentence endings
- Context awareness
- Robust handling of hesitation
- Robust handling of background noise
- Ability to handle different accents
- Ability to handle people speaking quickly or slowly
- Ability to recover gracefully when speech recognition is uncertain

The final system should feel like an actual person is participating in the conversation rather than an obvious automated IVR system.

MULTILINGUAL ROADMAP

Phase 1:

English

Phase 2:

Telugu

Phase 3:

Telugu + English code-switching / Tanglish

Phase 4:

Multiple Indian languages

Phase 5:

Other languages if technically and commercially justified.

Telugu is expected to be significantly more challenging because conversational Telugu requires good pronunciation, natural sentence rhythm, regional variation, code-switching, names, numbers and colloquial speech.

Do not assume that an English architecture will automatically produce high-quality Telugu.

We want to understand what must change between the English and Telugu versions.

RESEARCH QUESTIONS

I want you to independently investigate the entire problem and answer the following.

1. Telephony

What are the best current options for building an AI voice calling system in India?

Compare the major relevant providers and approaches rather than giving only two options.

Compare:

- Indian virtual numbers
- Cloud telephony providers
- CPaaS providers
- SIP providers
- Programmable voice APIs
- Real-time voice-AI-specific platforms
- SIM/eSIM-based approaches

Explain the advantages, disadvantages, pricing model, Indian number availability, KYC, DLT requirements, API quality, WebSocket/audio streaming capabilities and concurrency.

2. Real-time voice architecture

What are the critical components required to make a genuinely high-quality real-time voice agent?

Explain the important concepts without assuming a specific vendor:

- Telephony
- Audio streaming
- STT
- VAD
- Endpoint detection
- Turn-taking
- Barge-in/interruption handling
- Conversation orchestration
- LLM
- TTS
- Audio buffering
- Streaming
- Latency management
- Session management
- Memory
- Tool calling
- Call state
- WebSockets/SIP/RTP where relevant

Identify which components have the largest effect on perceived human quality.

3. Latency

What latency targets should we aim for?

Explain the complete latency pipeline from:

User stops speaking

→ speech detection

→ STT

→ reasoning

→ response generation

→ TTS

→ audio playback

Identify where latency normally occurs and how high-quality systems reduce perceived latency.

4. Interruption and turn-taking

This is one of the most important requirements.

Explain how high-quality voice agents handle:

- User interrupting the AI
- AI stopping immediately
- User pausing while thinking
- User speaking in incomplete sentences
- Simultaneous speech
- Background speech
- False interruption detection
- Long pauses
- Agent deciding when to respond

Explain what technologies and architectural patterns are normally used.

5. Voice quality

Research the current best approaches for achieving highly natural speech.

Compare current TTS approaches based on:

- Naturalness
- Latency
- Streaming
- Emotional expression
- Pronunciation
- Conversational quality
- English performance
- Indian accent performance
- Telugu capability
- Cost

Do not assume open-source models are automatically better or cheaper.

6. STT quality

Compare the best current speech recognition approaches for conversational phone audio.

Consider:

- English
- Indian accents
- Telephone audio
- Background noise
- Streaming
- Latency
- Accuracy
- Telugu
- Code-switching

7. LLM strategy

Determine whether a voice agent actually needs a large reasoning model.

Compare:

- Small/fast models
- Large models
- Local models
- Cloud models
- Hybrid architectures
- Multiple-model architectures

Explain where intelligence is actually required and where deterministic logic is preferable.

8. Cost

Estimate the realistic cost per minute of operating the system.

Break down:

- Indian telephone number
- Telephony
- STT
- LLM
- TTS
- Infrastructure
- Storage
- Recording
- Transcription
- Other relevant costs

Give approximate costs for:

- 100 calls/month
- 1,000 calls/month
- 10,000 calls/month
- 100,000 calls/month

Also compare the cost with traditional human calling.

9. Scalability

Explain how the system should handle:

- 1 concurrent call
- 10 concurrent calls
- 100 concurrent calls
- 1,000 concurrent calls

Identify the technical bottlenecks and provider limitations.

10. Indian telecom/legal considerations

Research the current Indian requirements for AI-generated outbound phone calls.

Specifically investigate:

- KYC
- DLT
- Caller ID
- Consent
- Promotional vs transactional calls
- Recording
- Privacy
- Data protection
- Automated calling restrictions
- Number provisioning
- Institutional/college use cases

Do not make legal assumptions. Use current authoritative sources wherever possible.

11. English → Telugu transition

Explain what will need to change when moving from English to Telugu.

Investigate:

- Telugu STT
- Telugu TTS

- Telugu pronunciation
- Regional dialects
- Code-switching
- Tanglish
- Names
- Numbers
- Locations
- Colloquial speech
- Conversational Telugu

Identify the biggest technical gaps in the current ecosystem.

12. Competitive landscape

Research current companies/products that are doing high-quality AI phone conversations.

Do not only list them.

Analyze what appears to differentiate the strongest systems in:

- Latency
- Voice quality
- Interruption handling
- Turn-taking
- Indian languages
- Telugu
- Telephony infrastructure
- Cost
- Developer flexibility

Do not assume that a company's marketing claims represent its actual technical advantage.

13. Critical risks

Identify the biggest reasons this project could fail.

I specifically want you to challenge the project on:

- Robotic voice
- High latency
- Poor interruption handling
- Bad turn detection
- Hallucinations
- Incorrect information extraction
- Telephone audio degradation

- Poor Indian accent recognition
- Telugu quality
- High per-minute costs
- Telecom restrictions
- Provider lock-in
- Concurrency limitations
- Unreliable call connections
- Poor scalability
- Privacy/security issues

14. Recommended technology strategy

Only AFTER independently researching all of the above, recommend:

1. Best telephony provider(s)
2. Best Indian number approach
3. Best STT approach
4. Best TTS approach
5. Best LLM strategy
6. Best real-time communication architecture
7. Best backend architecture
8. Best database
9. Best deployment approach
10. Best development roadmap

Give multiple alternatives where appropriate.

Do not recommend a technology merely because it is popular.

IMPORTANT

I am not asking you to immediately build the system.

I am currently trying to understand the **complete technical and commercial landscape** before development begins.

I want a research-grade answer based on current information.

Use current web sources wherever possible, especially official provider documentation and pricing pages.

Clearly distinguish between:

- Verified facts
- Provider claims

- Your technical inference
- Your recommendation

Most importantly, **challenge my assumptions instead of agreeing with them.**

The final objective is to build an English voice calling agent whose conversation quality is significantly better than a typical basic AI calling implementation, and then use the lessons from the English system to build a highly natural Telugu and eventually multilingual voice agent.

GLM 5.2 approach

1. Telephony: The Indian Landscape

Challenging Assumption: Do not assume that a normal SIM/eSIM is the correct architecture.

Verdict: A SIM/eSIM (even via gateways like Truphone or Twilio Mobile) is entirely the wrong approach for automated outbound calling in India. It lacks scalability, programmatic control, and violates TRAI's commercial usage guidelines for automated bulk calling.

Verified Facts & Provider Comparison (India):

Twilio (Global CPaaS):

Pros: Best developer experience, excellent WebSocket/Media Streams API for real-time bidirectional audio.

Cons: Indian numbers are difficult to provision (requires heavy KYC). Often routes calls via US/UK servers, introducing 150-300ms network latency before audio processing even begins.

Pricing: ~\$0.013/min outbound to India. Number rental ~\$1.15/mo.

Exotel / Kaleyra / Ozonetel (Indian CPaaS):

Pros: Deeply integrated with Indian telecom (Jio, Airtel, Vi). Excellent DLT compliance dashboards. Local data centers = low network latency.

Cons: Their core APIs are built for legacy IVR, not real-time AI. Many do not offer true bidirectional WebSocket streaming; they use POST APIs to trigger pre-recorded audio or basic dynamic TTS.

Pricing: ~₹0.80 - ₹1.50/min. Number rental ~₹200-₹300/mo.

Plivo / Sinch:

Pros: Sinch (via Voice API) and Plivo offer decent SIP trunking and WebSocket support. Plivo has good Indian coverage.

Cons: Still require strict DLT registration.

Voice-AI Specific Platforms (Vapi, Retell AI, Bland AI):

Pros: Out-of-the-box orchestration of STT, LLM, TTS, and telephony. You can bring your own Twilio/Plivo numbers.

Cons: High markup on usage (~\$0.10 - \$0.15/min). May not fully support Indian DLT routing natively.

Recommendation for Telephony: Do not build your own SIP/PBX infrastructure initially. Use Twilio Media Streams (bringing your own Indian virtual number via a local CPaaS provider that supports SIP forwarding, or using Twilio's Indian mobile numbers if you qualify). Alternatively, Exotel recently launched real-time WebSocket APIs—verify their latency. If you want ultimate control, rent an Indian SIP trunk from a local provider and bridge it to your application using FreeSWITCH or Asterisk.

2. Real-time Voice Architecture

To achieve "human-like" quality, you cannot use a sequential pipeline (Record -> STT -> LLM -> TTS -> Play). You must use a Streaming Event-Driven Architecture.

Critical Components:

Telephony Bridge: Connects PSTN to your server via WebSocket (RTP/Opus audio).

Audio Streaming: Must use 16kHz, 16-bit linear PCM. Phone lines are narrowband (8kHz mu-law), but upsampling to 16kHz gives STT models better accuracy.

VAD (Voice Activity Detection): WebRTC VAD or Silero VAD. Runs locally on the audio stream to detect speech presence.

Endpoint Detection: The hardest part. Determines when the user finished speaking. Relies on VAD silence thresholds + LLM intent parsing.

Turn-taking / Barge-in Orchestrator: A central state machine that tracks who is "holding the floor." If the AI is speaking and VAD detects user audio, it sends a "kill signal" to the TTS audio buffer.

Streaming STT: Continuously transcribes audio chunks.

LLM: Generates tokens in a stream.

Streaming TTS: Converts incoming LLM tokens to audio chunks before the LLM finishes generating the full sentence.

Largest Effect on Perceived Quality: VAD/Endpoint detection and Barge-in orchestration. A highly intelligent LLM with poor endpoint detection feels like a walkie-talkie conversation.

3. Latency

Human conversational latency (the gap between one person finishing and the other starting) is typically 200-400ms. Anything over 800ms feels like a delay.

Pipeline Breakdown & Targets:

Network Audio Transport (Telephony → Server): 40-80ms (Use AWS Mumbai ap-south-1 or local Indian DC).

VAD & Endpointing: 100-300ms (Requires waiting for silence to ensure the user actually stopped).

STT Processing: 100-200ms (Streaming STT returns text as soon as audio stops).

LLM Time-to-First-Token (TTFT): 150-300ms (Crucial. Must use fast inference providers).

TTS Time-to-First-Audio (TTFA): 100-200ms.

Audio Transport back to PSTN: 40-80ms.

Total Target: ~550ms - 1.1s.

How to reduce perceived latency further:

Speculative Execution: While the user is speaking, run an intent classifier on the partial transcripts. If the user is clearly saying "Yes", pre-generate the next logical response before the user even finishes.

Chunked TTS: Send TTS requests sentence-by-sentence, or even clause-by-clause.

Backchanneling: If endpointing takes longer than 1s, inject a quick "Hmm", "Right", or "I see" to fill the dead air.

4. Interruption and Turn-taking

Challenging Assumption: The system should stop speaking when the person interrupts.

Architectural reality: If you simply cut the audio upon hearing any voice, background noise, TV sounds, or the caller's own echo will constantly interrupt the AI, ruining the experience.

How High-Quality Systems Handle This:

Echo Cancellation: Hard on PSTN, but necessary.

Energy Thresholds + VAD: Background noise has different acoustic signatures than directed speech.

Semantic Barge-in: Instead of just VAD, use a fast STT model to transcribe the interrupting audio. If it detects words (not just noise), send the kill signal to the TTS buffer.

Handling Pauses/Hesitation: If the user pauses, STT yields a partial transcript. The LLM classifies: "Is this a complete thought?" If no, the orchestrator waits. If the pause exceeds 2 seconds, the AI can use a "floor-holding" phrase: "Take your time."

5. Voice Quality (TTS)

Challenging Assumption: Open-source models are automatically better or cheaper.

Verdict: False. Open-source models (like Bark, VITS, XTTS) require heavy GPU provisioning. At 100 concurrent calls, the cost of hosting real-time TTS GPUs often exceeds API costs, and latency is generally worse.

Current Best Approaches:

ElevenLabs: Best naturalness and emotion. Conversational latency is ~400ms. English is unmatched. Has Telugu support, but pronunciation of complex Telugu words/names can be Anglicized. Cost: ~\$0.30/1000 chars.

Cartesia (Sonic): Insanely fast (TTFA < 100ms). Excellent for conversational English. Highly recommended for low-latency barge-in. Cost: ~\$0.10/1000 chars.

PlayHT (Play3): Low latency, good cloning, decent Indian accents.

Google Cloud TTS / Azure TTS: Extremely reliable, excellent Telugu voices (specifically trained on native speakers). But sounds more "robotic" than ElevenLabs in conversational English.

6. STT Quality

For phone audio (8kHz), you must use models trained on telephone datasets.

Deepgram (Nova-2): The industry standard for speed and accuracy. Excellent on noisy audio. Strong Indian English accents. Latency < 150ms. Cost: ~\$0.0043/min.

AssemblyAI: Highly accurate, great semantic features (sentiment, speaker diarization), but slightly higher latency than Deepgram.

Google Speech-to-Text: Best for Telugu. Their multilingual models handle code-switching (Tanglish) natively better than Deepgram currently does.

OpenAI Whisper (API): Too slow for real-time barge-in. High hallucination rate on silence/noise.

7. LLM Strategy

Challenging Assumption: A voice agent actually needs a large reasoning model.

Verdict: False. GPT-4o, Claude 3.5 Sonnet, or o1 are overkill for a phone call. Their high TTFT (Time to First Token) ruins the conversation, and their reasoning capabilities lead to over-explaining.

The Strategy:

Core Dialogue Model: Use a fast, highly capable model. Llama 3.1 8B (via Groq or self-hosted), GPT-4o-mini, or Claude 3 Haiku. These have TTFT of < 200ms.

Tool Calling / Function Calling: Use the LLM only to extract structured data and determine conversation state (e.g., `absence_reason_captured: true`).

Deterministic Logic: For things like "Please spell your student ID," use deterministic string matching, not LLM parsing.

Hybrid Architecture: Use a fast classifier model to determine intent, and route the conversation. Only escalate to a heavier model if complex reasoning is strictly required (rare in outbound transactional calls).

8. Cost Analysis

Per-Minute Breakdown (English):

Telephony (India): ₹1.00 (\$0.012)

STT (Deepgram): ₹0.35 (\$0.004)

LLM (GPT-4o-mini / Llama 8B): ₹0.30 (\$0.004)

TTS (ElevenLabs/Cartesia): ₹0.50 (\$0.006)

Infrastructure/WebSockets: ₹0.20 (\$0.002)

Total per minute: ~₹2.35 (\$0.028)

Monthly Estimates (Assuming 3-min avg call duration):

100 calls/month (300 mins): ~₹705

1,000 calls/month (3,000 mins): ~₹7,050

10,000 calls/month (30,000 mins): ~₹70,500

100,000 calls/month (300,000 mins): ~₹590,000 (volume discounts apply)

Comparison to Human: A human BPO worker in India costs ~₹15,000 - ₹25,000/month plus overhead. A human can make ~50-80 calls a day. AI can make 10,000s a day. The ROI of AI is massive at scale.

9. Scalability

1 Concurrent Call: Can run on a single mid-tier VM.

10 Concurrent Calls: Needs a dedicated WebSocket server + external API integrations.

100 Concurrent Calls: Requires load-balanced WebSocket nodes. Deepgram/ElevenLabs handle concurrency easily. LLM API concurrency limits become the bottleneck here.

1,000 Concurrent Calls: Requires a microservices architecture. State management (Redis) is critical. SIP trunking concurrency limits (telephony provider) will require enterprise tier negotiations.

Bottlenecks: LLM rate limits (tokens per minute) and TTS concurrency limits.

10. Indian Telecom/Legal Considerations

Verified Facts (TRAI Regulations):

DLT (Distributed Ledger Technology): Mandatory in India. You must register your business, get a Header ID (Sender ID), and register the content template of your calls. Critical: AI

conversational scripts are hard to template because the exact words change. You must register a generalized transactional template.

Promotional vs. Transactional: You cannot use promotional routes for automated calls without strict consent mechanisms. College absence calling is arguably transactional (service/relationship based), which allows calling any time between 10 AM and 9 PM.

AI Disclosure: While specific AI calling laws are evolving, TRAI mandates that automated calls must not mislead. Using a human-sounding AI without disclosing it's an assistant is a legal gray area that is currently being scrutinized. Proceed with caution.

Caller ID: Must display the registered Indian virtual number.

11. English → Telugu Transition

Challenging Assumption: An English architecture will automatically produce high-quality Telugu.

Verdict: Completely false. Moving to Telugu breaks standard STT/TTS pipelines.

What must change:

Code-Switching (Tanglish): Users will say "Nenu illa unnanu, fever vastundi." (I am at home, getting a fever). Deepgram struggles with this natively. You must use Google Speech-to-Text with automatic language detection or multilingual models.

TTS: ElevenLabs Telugu often sounds like a foreigner reading Telugu phonetically. You must evaluate Azure Neural TTS (which has specifically trained Telugu voices like Shruti) or Bhashini (Indian government initiative) for natural Telugu.

Pronunciation: Telugu names (e.g., "D. Venkateshwarlu") and places will be butchered by standard TTS. You need custom pronunciation dictionaries (lexicons).

Grammar/Syntax: Telugu is Subject-Object-Verb. The LLM might generate English syntax translated to Telugu, which sounds robotic. The system prompt must enforce native Telugu syntax.

12. Competitive Landscape

Current Leaders:

Vapi.ai: Excellent orchestration. Fast, handles barge-in well. Highly developer-focused.

Skit.ai (Indian Company): Very strong in Indian context, debt collection, and multilingual support. They use custom acoustic models rather than just off-the-shelf APIs, giving them an edge in noisy Indian phone networks.

Retell AI: Great turn-taking models. They invested heavily in custom endpointing algorithms.

Gnani.ai (Indian): Strong local STT/TTS models built for Indian languages.

Differentiators: The strongest systems do not rely on standard OpenAI/Deepgram integrations. They build custom endpointing models trained on conversational data, and they use semantic barge-in rather than acoustic barge-in.

13. Critical Risks (Challenging the Project)

Bad Turn Detection: The project assumes the system can tell when a person finishes speaking. Over Indian mobile networks with high background noise, VAD will either cut users off constantly or hang in dead silence. Mitigation: Build a custom dual-model endpointer (VAD + fast LLM to validate if the sentence is complete).

Telephony Lock-in: If you build directly on Exotel's specific API, you cannot scale globally or switch providers if quality drops. Mitigation: Abstract the telephony layer. Use SIP/WebRTC bridges.

LLM Hallucination on Data Extraction: If the AI calls a student, and the student says "I will come on Monday," the LLM might extract "Next Monday" or hallucinate a date. Mitigation: Use strict JSON schemas and Tool Calling (Function Calling). Do not rely on free-text summarization for the final exported CSV.

DLT Blocking: TRAI is aggressive. If recipients mark your AI calls as spam, your virtual number will be blocked within days. Mitigation: Ensure the AI clearly states the purpose of the call within the first 3 seconds.

14. Recommended Technology Strategy

Based on the independent research above, here is the optimal path forward:

1. Telephony Provider:

Prototype: Twilio (for rapid WebSocket Media Streams dev) OR Exotel (if Indian number DLT is the immediate priority).

Scale: Rent an Indian SIP trunk from a local provider (e.g., Jio/Airtel enterprise) and bridge via FreeSWITCH.

2. STT:

English: Deepgram (Nova-2 conversational model). Unbeatable speed.

Telugu/Tanglish: Google Speech-to-Text (V1 API with auto_decoding and multilingual config).

3. TTS:

English: Cartesia Sonic (for lowest latency) or ElevenLabs (for best naturalness).

Telugu: Microsoft Azure Neural TTS (Telugu voices are far superior to ElevenLabs for native pronunciation).

4. LLM:

Llama 3.1 8B (served via Groq for ultra-fast TTFT) or GPT-4o-mini. Keep prompts incredibly short. Use JSON tool calling for data extraction.

5. Real-time Architecture:

Do not build polling loops. Use an event-driven WebSocket orchestrator. As audio streams in, push it to STT. As STT yields text, push to a state machine. If state == "user_turn_complete", stream prompt to LLM. Stream LLM tokens to TTS. Stream TTS chunks back to the WebSocket. Implement a "barge-in" event bus: if VAD triggers during "ai_speaking" state, send a kill signal to the TTS stream.

6. Backend Architecture:

Node.js or Go for the real-time WebSocket layer (critical for I/O concurrency).

Python/FastAPI for the LLM/STT integration and business logic.

Redis for ephemeral call state (who is speaking, current prompt context).

Celery/RabbitMQ for background tasks (transcription, summary generation, CSV export).

7. Database:

PostgreSQL for relational data (Campaigns, Contacts, Call Logs, Results).

8. Deployment:

AWS Mumbai (ap-south-1). This is non-negotiable for latency. Hosting in US/EU will add 250ms+ of network latency, ruining the human feel.

9. Development Roadmap:

Phase 1 (Weeks 1-3): Text-to-Speech conversational loop in terminal. Master the VAD, STT, and TTS pipeline before touching telephony. Aim for <800ms latency.

Phase 2 (Weeks 4-6): Integrate Twilio/Exotel WebSocket. Handle real PSTN audio (mu-law to PCM conversion). Implement barge-in.

Phase 3 (Weeks 7-8): Implement the LLM state machine, tool calling for student data extraction, and campaign management DB.

Phase 4: Begin Telugu evaluation. Swap Deepgram for Google STT, ElevenLabs for Azure. Evaluate code-switching failure rates.

k3 approach

1. Telephony: Best Current Options for AI Voice Calling in India

The Core Architectural Decision: SIM/eSIM vs. Cloud Telephony vs. SIP

Your assumption that a "proper Indian telephone number" requires evaluating SIM/eSIM is worth challenging. For a scalable AI voice platform, a physical SIM or eSIM is almost never the correct architecture. Here is why:

Table

Approach	What It Is	Pros	Cons	Verdict
Physical SIM / eSIM	Consumer mobile phone in a GSM network or Android phone	Very low per-minute cost; no KYC beyond initial SIM	Illegal/gray-area for commercial bulk calling; no API control; no concurrent calls; TRAI considers this "gray area"; no call recording; no agent automation; service failure = downtime; no CLI management	Inappropriate
Cloud Telephony (CPaaS)	Exotel, Ozonetel, Wowlar, Plivo, Twilio	Full API control; Websocket/RTP streaming; DLT compliance support; concurrent scaling; legitimate CLI	Higher per-minute cost; KYC required; monthly commitments	Correct for prototypes and mid-scale
Direct SIP Link	Direct carrier relationship with Airtel Business, Jio Business, Tata	Lowest per-minute cost at scale; best efficiency; full control	You become your own CPaaS; need SBC, config, codec negotiation, DLT licensing, telecom engineer	Correct for 20L+ minutes/month
Hybrid	Direct SIP for volume + CPaaS for overflow/failover	Cost optimization + operational safety	Complexity	Correct for mature platform

Verified fact: Direct SIP trunking with Airtel/Jio/Tata at 20 lakh+ minutes/month drops outbound rates to ₹0.60–₹1.20/min versus ₹0.80–₹1.80 through aggregators.

Technical inference: For your prototype and early production (under ~10,000 calls/month), a CPaaS provider is the only rational choice. A SIM-based approach would require you to build hardware redundancy, call rotation to avoid carrier spam flags, and manual DND scrubbing — all of which a CPaaS handles.

Provider Comparison for India (2026)

Table

Provider	India Focus	Outbound ₹/min	DID Provisioning	DLT Support	WebSockets / RTP Streaming	Best For
Exotel	Very High	₹0.80–1.00	2–5 days	Strongest operational support	AgentStream (<50ms)	India-native voice AI; long hand-holding
Ozonet	Very High	Competitive	3–7 days	Strong	Yes	Contact-center workflows; unlimited minute plans
Knowlar	High	Competitive	2–5 days	Strong	Yes	Mid-market with compliance needs
Plivo	Moderate	₹0.80–1.20	24–48 hours	Competent	Yes	Developer-led; global expansion later
Twilio	Low India	₹1.20–1.50	24–48 hours	Thinnest	Yes	Global-first teams; lack India-specific compliance
Direct (Airtel/Jio)	N/A	₹0.60–1.20	Week	You handle	Raw RTP	20L+ minutes/month; need telecom engineer

Verified fact: Exotel's pricing is typically ₹0.80–1.00 for outbound and ₹0.30–0.50 for inbound, with AgentStream providing <50ms media latency for AI streaming.

Verified fact: Plivo and Twilio provision new Indian DID numbers in 24–48 hours versus 2–7 business days for India-native providers.

Challenge to your assumption: Twilio is not the best choice for India despite its global popularity. Its India DLT support is the thinnest among major providers, and its India outbound rates are 20–50% higher than Exotel/Plivo. If you start with Twilio because "it's what everyone uses," you will pay a premium and struggle with TRAI compliance.

KYC, DLT, and Regulatory Requirements

DLT Registration (for SMS): If your system sends any SMS (appointment reminders, OTPs, consent confirmations), you must register on a TRAI-approved DLT platform. This is a three-phase process: Entity Registration → Sender ID Registration → Template Registration. Cost: ₹5,000 + GST for entity registration; ₹50,000 + GST for telemarketer delivery function.

For voice calls specifically: The DLT framework is primarily SMS-oriented, but TRAI's TCCCPR regulations and the new Digital Consent Registry apply to voice communications as well. You must:

- Register as a telemarketer if making outbound commercial calls
- Scrub against DND (Do Not Disturb) lists
- Maintain call logs for 1 year
- Implement recording disclosure
- Restrict calling hours (typically 9 AM – 9 PM for marketing; 9 AM – 7 PM for BFSI)

Critical 2026 update: TRAI has introduced mandatory AI disclosure for AI-generated voice calls. Your agent must clearly state at the beginning of the call that it is AI-powered, the purpose, company identification, and how to opt out. Penalties for non-compliance reach up to ₹10 lakh, with repeat offenders facing disconnection and blacklisting.

Number series restrictions:

- 140 series: Strictly for marketing/promotional calls
- 160 series: Reserved for transactional/service calls from regulated entities (RBI, SEBI, IRDAI, PFRDA)
- Cross-selling during service calls on 160 series is prohibited

For your college absentee use case: This is arguably a transactional/service call (educational institution contacting enrolled students about attendance). You may qualify for 160-series numbers or standard DID's, but you still need explicit consent or a legitimate educational relationship. Do not assume "we're a college" exempts you from consent requirements.

2. Real-Time Voice Architecture

The architecture is a cascaded pipeline with intelligent orchestration, not a single model. Here are the critical components and their quality impact:

Component Breakdown

Table

Component	Function	Quality Impact	Typical latency
Telephony	PSTN/SIP connectivity, audio capture	Medium — 8kHz μ -law grades audio	20–100ms work

VAD (Voice Activity Detection)	Detects if human is speaking	Critical — bad VAD = false interruptions or delayed responses	Runs continuously
STT (Speech-to-Text)	Converts speech to text	Critical — errors cascade to LLM	50–300ms (streaming)
Endpoint Detection	Decides when user is finished speaking	Critical — hardest problem in voice AI	100–400ms
LLM	Reasoning, response generation	High — but latency matters more than size	100–500ms (TFT)
TTS (Text-to-Speech)	Converts response to audio	Critical — dominant cost and perceived quality	40–200ms (FB)
Turn-Taking Orchestrator	Coordinates who speaks when	Critical — defines "naturalness"	Orchestrator layer
Barge-In Handler	Stops AI when user interrupts	Critical — worst failure mode is talking over user	<100ms pop time
Session Manager	Maintains call state, text, memory	High — prevents repetition, maintains flow	N/A
Tool Calling	Extracts structured data, triggers actions	Medium — latency spikes if asynchronous	100–500ms

The Streaming Principle

The single most important architectural concept: Every stage must stream into the next without waiting for completion.

- STT emits partial transcripts every ~50ms
- LLM streams tokens as they generate
- TTS streams audio chunks before the full response exists
- Endpoint detection uses a small neural model updating probability dozens of times per second

Technical inference: A non-streaming architecture adds 2–4 seconds of perceived latency. A streaming architecture can achieve ~600ms end-to-end.

What Actually Differentiates Production-Grade Systems

In 2026, the raw STT/LLM/TTS components are commoditized. What separates "demo" from "production-grade" is:

- 1. Endpoint detection quality — A naive timeout (e.g., 1.5s silence) destroys naturalness. Humans pause mid-sentence, say "um," and breathe. The 2026 answer is a trained conversational endpoint model.
 - 2. Barge-in recovery — When interrupted, the system must stop TTS within 100ms, discard the remaining planned response, and start a fresh STT stream without losing context.
 - 3. VAD tuning — Semantic VAD (understanding meaningful speech vs. background noise) outperforms energy-based VAD.
 - 4. Turn-taking model — A dedicated small model that decides when the human has finished, considering prosody, semantics, and conversational history.
-

3. Latency Targets

The Complete Latency Pipeline

Table

Stage	Target	Where Latency occurs	Optimization
Network round-trip	20–80ms	Physical distance to server	Edge deployment (Mumbai/Bangalore)
VAD detection	10–50ms	Energy vs. semantic detection	Use semantic VAD (neural)
Endpoint detection	100–300ms	Conservative timeouts	Train/fine-tune endpoint model on conversational data
STT	50–200ms	Waiting for complete utterance	Streaming STT with partial transcripts
LLM TTFT	100–400ms	Large model inference time	Use small/fast model; streaming; speculative decoding
TTS TTFB	40–200ms	First audio chunk generation	Streaming TTS; optimize for speed
Audio playback	20–50ms	Buffering	Minimal buffering; WebRTC if possible
TOTAL	<700ms		

Verified fact: Under ~700ms end-to-end response time is the threshold where conversation feels human. Above that, callers interrupt, repeat themselves, and hang up.

Where latency actually occurs: Most latency hides in turn-taking decisions and LLM time-to-first-token, not STT or TTS. A typical budget: network 50ms, VAD/turn-taking 200ms, LLM TTFT 250ms, TTS first audio 100ms. STT runs largely in parallel with speech.

How High-Quality Systems Reduce Perceived Latency

1. Predictive TTS: Start generating audio for high-probability responses before the LLM finishes.
2. Filler audio: Inject "um," "hmm," or "let me check" while tools run.
3. Progressive responses: Speak the first clause while generating the rest.
4. Edge deployment: Run VAD/STT/endpoint models in Mumbai/Bangalore, not US-East.
5. Speculative execution: Pre-generate likely follow-up responses.

4. Interruption and Turn-Taking

This is the most technically difficult requirement and the primary differentiator between robotic and natural systems.

Handling Specific Scenarios

Table

Scenario	Technique	Common Failure
User interrupts	Barge-in detection → immediate TTS stop → flush buffer → new STT stream	TTS continues for 500ms+; stale response plays
AI stopping immediately	<100ms TTS cancellation; discard already played audio chunks	Partial word cutoff sounds robotic
User pausing while thinking	Semantic endpoint model distinguishes "thinking pause" from "finished"	Naive timeout cuts off response or delays response
Incomplete sentences	LLM trained on conversational repair; handles clarifying questions	AI treats incomplete input as complete
Simultaneous speech	Crosstalk detection; defer to user; mark overlapping audio	Both speakers continue; garbled audio

Background noise	Directional/noise-robust VAD; spectral subtraction	False trigger from radio
False interruption detection	Hysteresis in barge-in threshold; confirm with 200ms lookahead	AI stops for every cough
Long pauses	Progressive "are you there?" prompts; comfort noise	Dead silence feels abandoned
Agent decides when to respond	Turn-taking probability model combining acoustic + lexical + conversational features	Always responds too fast or too slow

Architectural Patterns

The 2026 standard pattern: A small neural turn-taking model (often ~100M parameters) runs on the audio stream, outputting a probability of "user has finished" dozens of times per second. This model is trained on real conversational telephone audio.

Barge-in implementation: The system maintains two audio streams:

1. Agent output stream (TTS → telephony)
2. User input stream (telephony → VAD → STT)

When VAD detects user speech during agent output, a barge-in event fires. The orchestrator must:

1. Send TTS stop signal (<50ms)
2. Flush remaining TTS buffer
3. Reset STT stream (or maintain separate context)
4. Start new inference turn

Technical inference: Poor barge-in handling is the #1 reason users perceive voice AI as "robotic." A system that talks over the user for even 300ms feels fundamentally broken.

5. Voice Quality (TTS)

Current TTS Comparison (2026)

Table

Pro vider	Model	Naturalne ness	Late st (GPT-4o FB)	Stre ngths	India accent	Telu gu	Cost (\$/hrs)	Best For

ElevenLabs	v3 / v2.5	Excellent (ELO)	~100	Yes	Good (English)	Limit	~\$1	Premium English; voice cloning
Cartesia	Sonic-3	Very Good	~40–50ms	Yes	Moderate	No	~\$3	Ultra-low latency
Inworld	TTS-2 / S 1.5 Max	Excellent (~2 ELO)	~120–150ms	Yes	Good	No	~\$10–35	Expressive; long runtime; scalable
Google	Gemini Flash	Excellent (ELO)	Variable	Yes	Good	Good	~\$30	Multilingual; Indian languages
DesiVocal	Aura-2	Good	~313	Yes	Moderate	No	~\$10	Integrated; cost
VoisLabs	Default	Good (Indian languages)	Moderate	Batch/Streaming	Good	Native quality	~₹8.00/min	Indian languages; primary
DesiVocal	Default	Moderate	Moderate	Yes	Good	Yes	~₹300/mo	Indian-built; ₹ billing
OpenAI	gpt-4o-mini-tts	Good	Moderate	Yes	Moderate	No	Bundled	OpenAI ecosystem

Verified fact: Inworld TTS 1.5 Max ranks #1 on Artificial Analysis Speech Arena ELO (1,208), followed by Google Gemini 3.1 Flash TTS (1,206), with ElevenLabs v3 at #4 (1,178).

Verified fact: For Indian languages specifically, VoisLabs and Narakeet lead. VoisLabs offers native Telugu voices, English→Telugu transliteration, and code-mixed ("Tanglish") handling. ElevenLabs and Murf are over-indexed for this market at their current price points.

Challenge to Your Assumption

Do not assume open-source TTS is better or cheaper. Open-source models like Coqui TTS, Piper, or XTTS require significant GPU infrastructure, voice cloning data, and tuning to sound natural. For a small team, the engineering cost exceeds API costs. Commercial APIs also update models continuously — a hidden operational cost of self-hosting.

For English: ElevenLabs, Inworld, and Google Gemini Flash TTS are the top tier. Cartesia wins on raw latency but sacrifices some expressiveness.

For Telugu: This is where the ecosystem gaps appear. ElevenLabs has Telugu voices but quality is described as "limited" for conversational use. VoisLabs and DesiVocal are specifically built for Indian languages. Google Cloud TTS has Telugu support but conversational naturalness is

questionable. The biggest gap: No major realtime TTS provider offers truly natural, conversational Telugu with emotional range and code-switching.

6. STT Quality

Comparison for Conversational Phone Audio

Table

Provi	Mode	Englis WER	Phon audio WER	Indi cent	Te u	Strea ing ency	Cost/min	Best For
Deep m	Nova	~5.26 (batch)	21.8% allHome	Goo	P	<300	\$0.0043	Real-time ce agents; ed
Deep m	Flux	N/A	15.58 (altime)	Goo	P	<300	\$0.0065	Conversatio agents
Asse lyAI	Unive l-3.5	~7.0% gregate	~23.4	Goo	N	~300	\$0.0035	Max uracy; st-processing
Whisp denAI)	Large	~8–12 l-world	~26.4	Goo	~ -28	1–5s (tch)	Free (f-host) / 006	Multilingual; ian guages
Googl	Chirp	~3–4 clean	~7–10 noisy	Ver ood	G	~350	\$0.006	Indian guages
Eleve abs	Scrib 2	Good	Good	Goo	U ow	Realti	\$0.0065	Integrated h ElevenLabs S

Verified fact: On conversational phone audio (CallHome benchmark), Deepgram Nova-3 achieves 21.8% WER versus Whisper's 26.4% and AssemblyAI's 23.4%. Even the best engine misses roughly 1 word in 5 on phone audio.

Verified fact: Whisper Large-v3 achieves ~21.5% WER on Hindi FLEURS benchmark. Telugu WER on Whisper (from academic research) is approximately 20.3–26.7% depending on model size and prompting.

Critical finding for Indian accents: The Voice of India benchmark found that best-performing models scored 5–6% WER on Indo-Aryan languages (Hindi, Bengali) but 15–20% on Dravidian languages (Tamil, Telugu, Malayalam).

Challenge to your assumption: Deepgram is the dominant choice for English voice agents due to speed and cost, but its multilingual accuracy is significantly worse than alternatives. For Telugu, Google Cloud STT and Whisper are better starting points, though all options have substantial quality gaps for conversational Telugu.

Code-Switching (Tanglish)

This is a major unsolved problem. When speakers switch between Telugu and English mid-sentence:

- Deepgram Nova-3 Multilingual achieves 12.22% WER on code-switching audio
- AssemblyAI Universal-3.5 Pro achieves 7.69% WER on code-switching
- Whisper handles Hinglish/Tanglish better than most commercial APIs due to its diverse training data, but still struggles with mid-sentence switches

Technical inference: For Phase 3 (Telugu + English code-switching), you will likely need a custom solution or significant prompt engineering. No off-the-shelf API handles Tanglish conversationally at production quality.

7. LLM Strategy

Do You Need a Large Reasoning Model?

Short answer: No, not for most turns.

A voice agent does not need frontier-level reasoning for every response. The LLM's job in a voice conversation is:

1. Understand intent (classification)
2. Maintain context (short-term memory)
3. Generate natural, concise responses (not essays)
4. Decide when to call tools (structured extraction)
5. Handle edge cases (escalation, clarification)

Model Architecture Options

Table

Architecture	Description	Latency	Cost	When to Use
Small fast del (3B–9B)	Gemma 4 26B, Qwen3,stral Small	Very low	Very low	90% of turns; fine conversation

Medium del (B–70B)	GPT-4.1, Claude Sonnet, mini Flash	Lo	Moderate	Complex reasoning; Multi-step extraction
Large del (400B+)	GPT-5.5, Claude Opus, mini Pro	Hig	High	Escalation only; ge cases
Hybrid ting	Small model handles routine; ge model handles complex	Opt zed	Optimize	Recommended hitecture
Speech-to-s ach native	GPT-Realtime 2.1, Gemini e	Lo	Very high (.09/min)	When single-vendor plicity matters

Verified fact: A small open model like Gemma 4 26B costs under \$0.0002/min in LLM tokens, while GPT-5.5 at \$5 input/\$30 output per 1M tokens adds ~\$0.015/min — more than an entire cascaded stack with a small model.

Recommendation: Use a hybrid routing architecture. Route 90% of turns to a fast, cheap small model (Gemma 4, Qwen3, or Mistral Small). Escalate to a larger model only for:

- Ambiguous intent classification
- Complex structured extraction
- Sentiment analysis requiring nuance
- Tool call failures

This reduces both latency and cost by 80%+ without sacrificing quality.

Deterministic Logic vs. LLM

Not everything needs an LLM:

- Call status tracking (answered, busy, failed) → Deterministic telephony webhook
- DND scrubbing → Deterministic API call
- Simple information collection (name, date) → Can use deterministic grammar + LLM

validation

- Transcript generation → Deterministic STT output

Use LLMs for language understanding and generation only. Everything else should be deterministic code.

8. Cost Estimation

Component Cost Breakdown (India, per conversation minute)

Table

Component	Low-Cost Option	Mid-Range	Premium	Notes
Telephony (dia outbound)	₹0.60 (direct P)	₹0.90 (otel/Plivo)	₹1.50 (ilio)	Dominant cost scale
STT	\$0.0036 (Deepgram Nova-2)	\$0.0043 (Deepgram Nova-3)	\$0.0075 (AssemblyAI)	Billed per audio minute
LLM	\$0.0002 (small open model)	\$0.002 (GPT-4.1)	\$0.015 (GPT-5.5)	Token-based; privacy matters
TTS	\$0.003 (Deepgram Aura-2)	\$0.005 (Cartesia)	\$0.010 (ElevenLabs)	Dominant API cost
Orchestration	\$0 (DIY)	\$0.03 (SigmaMind)	\$0.055 (Exotel)	Platform fee if used
Storage/Record	\$0.001	\$0.002	\$0.005	S3/Cloudflare
Infrastructure	\$0.001	\$0.002	\$0.005	Compute, load balancers

Verified fact: TTS dominates every cascaded stack at 70–73% of raw component cost. The LLM is under 4% if using a small open model.

Total Cost Per Minute (All-In)

Table

Scenario	Cost/Min	Notes
DIY low-cost stack	~\$0.007–0.0	Deepgram + Gemma 4 + Deepgram Aura + Exotel SIP
DIY mid-range stack	~\$0.025–0.0	Deepgram + GPT-4.1 + Cartesia + Exotel
DIY premium stack	~\$0.060–0.0	AssemblyAI + GPT-4.1 + ElevenLabs + Exotel
Managed platform (and)	~\$0.09–0.14	All-inclusive; cheapest managed option

Managed platform (tel)	~\$0.13–0.31	Component-based; transparent
Managed platform (pi)	~\$0.12–0.25	\$0.05 platform + provider costs
Speech-to-speech (enAI)	~\$0.09	GPT-Realtime 2.1; single vendor

Monthly Cost at Scale (assuming 3 min avg call duration)

Table

Volume	DIY Low	DIY Mid	Managed (tel)	Human Agent (ivalent)
100 calls/mo (300 min)	~\$2–5	~\$8–12	~\$40–90	~\$150–300
1,000 calls/mo (3,000 min)	~\$20–45	~\$75–120	~\$400–900	~\$1,500–3,000
10,000 calls/mo (30,000 min)	~\$200–450	~\$750–1,200	~\$4,000–9,000	~\$15,000–30,000
100,000 calls/mo (300,000 min)	~\$2,000–5,000	~\$7,500–15,000	~\$40,000–90,000	~\$150,000–300,000

Challenge to your assumption: Human calling in India costs \$5–16 per agent hour offshore. At 3-minute calls, a human agent handles ~20 calls/hour = \$0.25–0.80 per call minute in labor alone, plus infrastructure, management, and turnover costs. AI voice agents at \$0.07–0.15/min are already cost-competitive, but the gap narrows at lower volumes where human agents can multitask.

Hidden costs not in the table:

- Barge-in waste: 10–30% extra TTS cost from poor interruption handling
- LLM history growth: 30-minute calls can 2–3× LLM token costs
- Minimum commitments: Deepgram Growth requires \$4K+/year; ElevenLabs tiers have unused quota waste
- DLT registration and compliance ongoing costs

9. Scalability

Scaling Scenarios

Table

Con rent lls	Architecture	Bottlenecks	Provider Limits
1	Single WebSocket thread; al dev	None	N/A
10	Single server; async event p	LLM rate limits; TTS currency	Most APIs handle 10 sily
100	Load-balanced workers; dis for state	Telephony trunk capacity; STT socket limits; M TPM	Exotel/Plivo: negotiate ; Deepgram: concurrency s
1,00	Kubernetes cluster; licated SBC; multi-region	Telephony trunk uration; TTS throughput; abase writes	Requires direct SIP or erprise CPaaS; custom astructure
10,0	Multi-region auto-scaling; ect carrier relationships; ge compute	Carrier-grade routing; ulatory limits; cost losion	Requires direct Airtel/Jio ationships; dedicated astructure

Technical inference: The primary bottleneck at 100+ concurrent calls is telephony trunk capacity, not AI components. A CPaaS provider will enforce a concurrent-call ceiling. At 1,000 concurrent calls, you need either:

1. Enterprise CPaaS with burst allowances
2. Direct SIP trunking with multiple carriers
3. A hybrid approach (primary + overflow carriers)

Database consideration: At 100+ concurrent calls, your database must handle high write throughput for call state updates. Use Redis for hot session state (sub-millisecond latency) and PostgreSQL or ClickHouse for persistent call records and transcripts.

10. Indian Telecom/Legal Considerations

Current Regulatory Framework (2026)

TRAI TCCCPR Amendment (February 2026)

Verified fact: TRAI's February 2026 amendment introduced:

- 1. Mandatory AI disclosure at call opening
- 2. Digital Consent Registry — real-time verification before dialing
- 3. 140/160 series number segregation — marketing vs. service
- 4. Penalties up to ₹10 lakh for violations; repeat offenders face 1-year disconnection and blacklisting

Specific Requirements for Your Use Case (College Absentee Calling)

Table

Requirement	Status	Action Needed
KYC	Mandatory	Business PAN, GST, address proof
DLT Registration	Mandatory if any S	Entity + Header + Template registration
Telemarketer gistration	Likely required	Register as principal entity with TRAI
Consent	Required	Students must have consented to contact (rollment forms)
DND Scrubbing	Mandatory	Scrub against National DND registry
AI Disclosure	Mandatory 26)	Script must disclose AI nature
Recording closure	Mandatory	Announce "this call is recorded"
Calling Hours	Restricted	9 AM – 9 PM for most; 9 AM – 7 PM for BFSI
Call Log Retention	1 year	Maintain records

Data Protection	DPDPA 2023 plies	Consent withdrawal mechanism; data imization
-----------------	---------------------	---

Critical challenge: Your "college absentee" use case sits in a gray area. Is it transactional (educational institution contacting enrolled students about attendance) or commercial (if any survey/data collection beyond immediate academic need)? If transactional, you may use standard DIDs. If commercial/marketing, you need 140-series numbers and explicit consent.

Do not assume institutional use exempts you. TRAI's regulations apply to all automated calling. The "educational" exemption is narrower than most assume.

Academic/Student Access

Most providers (Twilio, Exotel, Plivo) offer startup credits but not specific academic discounts for telephony. You can typically get:

- Exotel: 7-day trial with ₹500 credits
- Twilio: \$15–50 trial credit
- Plivo: Trial credits on request

For significant volume, negotiate as a startup rather than an academic project.

11. English → Telugu Transition

What Must Change

Table

Component	English Approach	Telugu Requirement	Gap Severity
STT	Deepgram v3 or isper	Google Cloud STT or isper; custom fine-tuning likely needed	Critical — 15–20% WER on Dravidian languages vs. 5–6% on Indo-Aryan
TTS	ElevenLabs Resemble/Inworld	VoisLabs, DesiVocal, or Google Cloud	Critical — No premium realtime Telugu TTS exists
Endpoint Detection	English-trained models	Needs Telugu acoustic/prosodic training	High — Pauses, fillers, and turn phases differ
LLM Prompting	English system prompts	Telugu prompts; context-switching handling	High — Tanglish is not well-handled by any API

Pronunciation	Standard English phonemes	Telugu retroflexes, vowel length, sandhi	Critical — Names, places, numbers often mispronounced
Region Dialects	Minor accent variation	Telangana vs. Andhra dialects; regional vocabulary	High — Models trained on standard Telugu
Code-switching	Occasional words	Heavy Tanglish (Telugu + English mid-sentence)	Critical — Major unsolved problem
Numbers	Standard English number system	Telugu number system; lakhs/crores; ordinal forms	Moderate — TTS often falls back to English numerals
Names	Standard pronunciation	Telugu names with Sanskrit/Telugu origin; honorifics	High — Critical for your specific use case

The Biggest Technical Gaps

1. Conversational Telugu TTS: Current TTS for Telugu is either robotic (Google Cloud) or not realtime (VoisLabs is batch/streaming hybrid). There is no ElevenLabs-quality Telugu voice that streams with <200ms latency.
2. Telugu STT on Phone Audio: Whisper achieves ~20–27% WER on Telugu. On 8kHz phone audio with background noise, this degrades to potentially 30–40% — borderline unusable for information extraction.
3. Code-Switching: No production API handles Telugu-English code-switching reliably. AssemblyAI's 7.69% WER on code-switching is for major European languages, not Tanglish.
4. Colloquial Speech: Telugu conversational fillers ("ante," "emito," "inkenti"), discourse markers, and informal grammar are poorly represented in training data.

Recommendation: Do not attempt Telugu until your English system has:

- Proven interruption handling
- Proven structured extraction accuracy >90%
- Proven endpoint detection quality
- A clear path to custom model fine-tuning

12. Competitive Landscape

Major Players (2026)

Table

Company	Differentiation	Latency	Voice quality	Indian languages	Telephony	Cost	Developer experience
Indico AI	#1 TTS quality; model-agnostic (220+ LLMs); expressive steering	~100ms	Excellent	15 GA; experimental	BY	Low scale	High
Retell AI	Native telephony; enterprise compliance; structured dialog	~60ms	Very good	Moderate	Buy	\$0.1331/min	Moderate
Vapi	Provider-agnostic orchestration; 14+ providers	Variable	Good (depends on providers)	Dependent on providers	BY	\$0.05/providers	High
Bland AI	Cheapest at scale; inbound-native; campaign management	Good	Good (low-mid)	Limited	Buy	\$0.0914/min	Low
Cosmos AI	Ultra-low latency TTS (10ms TTFB); full stack (Voice + Ink + Line)	~40ms TTFB	Very good	42+ languages	Buy	\$0.06/telephony	Moderate
ElevenLabs	Broadest language coverage; voice cloning; devAI platform	~100ms	Excellent	70+ languages	Buy	High	Moderate
Bolna AI	Indian/South Asian market focus; INR pricing	Good	Moderate	Indian languages	Inbound-native	\$0.0507/min	Moderate
OpenAI	Unified ecosystem; realtime API; SIP connectivity	Low	Good (not mid)	57+ languages	Buy	\$0.09/min	Low (hook-in)
Dogra	Integrated speech stack; Voice Agent API; turn prediction	Low	Good (para-2)	36+ languages	Buy	\$0.0516/min	Moderate

Verified fact: Bland AI is optimized for high-volume outbound at ~\$0.09/minute all-in, making it 30–50% cheaper than Vapi or Retell at scale. Voice quality sits below Vapi/ElevenLabs-based platforms.

Verified fact: Bolna AI specifically targets Indian/South Asian language markets at \$0.05–0.07/min.

What Differentiates the Strongest Systems

The strongest systems (Inworld, Retell, top-tier custom stacks) differentiate on:

- 1. Endpoint model quality — Not raw TTS latency, but when to speak
- 2. Barge-in recovery speed — <100ms TTS cancellation
- 3. Expressive voice control — Natural pauses, breathing, emotional range
- 4. Turn-taking orchestration — Not just fast components, but correct sequencing

Marketing claims vs. reality: Many platforms claim "human-like conversations." In practice, most implementations still feel robotic because:

- They use naive timeout-based endpoint detection
- They don't handle barge-in gracefully
- They use cheap TTS that sounds "good enough" in demos but grating over 10+ minutes
- They don't vary prosody based on conversational context

13. Critical Risks

Table

Risk	Likelihood	Impact	Mitigation
Robotic voice	Medium	High	Invest in premium TTS (ElevenLabs/Inworld); use expressive steering; avoid cheap bundled voices
High latency	High	High	Edge deploy in India; use small LLM for 90% of requests; optimize turn-taking model
Poor interruption handling	High	Critical	This is the #1 failure mode. Test barge-in aggressively. Budget 10–30% TTS waste.
Bad turn detection	High	High	Do not use naive timeouts. Use semantic endpoint detection.
Hallucinations	Medium	High	Use deterministic logic for extraction; LLM only for conversation; validation layers
Incorrect information extraction	Medium	High	Multi-pass extraction; validation prompts; human review for critical fields

Telephone audio degradation	Certain	Medium	8kHz μ -law is unavoidable; train/test on phone audio specifically; use noise-robust STT
Poor Indian accent recognition	Medium	High	Test Deepgram vs. Whisper vs. Google on your target accent; consider fine-tuning
Telugu quality	Certain	Critical	No production-ready conversational Telugu stack exists. This is a research project, not a configuration.
High per-minute costs	Medium	Medium	TTS dominates; negotiate committed tiers; use all LLM; direct SIP at scale
Telecom restrictions	Medium	Critical	TRAI can shut down campaigns in 24 hours. Full compliance from day one.
Provider lock-in	Medium	Medium	Use modular architecture (Vapi-style or DIY); avoid speech-to-speech native models
Concurrency limitations	Medium	High	Negotiate burst allowances; plan for direct SIP 1000+ concurrent
Unreliable connections	Medium	Medium	Multi-carrier failover; retry logic; call quality monitoring
Poor scalability	Low	High	Stateless design; Redis for session state; horizontal scaling
Privacy/security	Medium	High	DPDPA compliance; encryption; minimal data retention; consent audit trail

Biggest risk to project success: The combination of poor interruption handling + high latency + robotic voice creates a death spiral where users hang up immediately. You can have perfect information extraction, but if the conversation feels robotic, nobody will stay on the line long enough to extract anything.

Second biggest risk: TRAI compliance. A single DND violation or missing AI disclosure can result in number blacklisting, which kills your entire operation.

14. Recommended Technology Strategy

Based on independent research, here are my recommendations. I provide alternatives where appropriate.

1. Best Telephony Provider(s)

Primary recommendation: Exotel

- Best India-native DLT compliance support
- AgentStream for <50ms media latency
- Competitive ₹0.80–1.00/min outbound
- Strong operational support for Indian regulations

Alternative: Plivo

- Better for globally-minded teams
- Faster DID provisioning (24–48 hours)
- Good API quality
- Slightly weaker India compliance support

Avoid: Twilio for India-specific deployment

- Thinnest DLT support
- Highest India outbound rates
- Better suited for global expansion later

At scale (>20L min/month): Direct SIP with Airtel Business or Jio Business, using Exotel/Plivo as CPaaS layer for API convenience.

2. Best Indian Number Approach

- Start with standard DID numbers from Exotel (not toll-free)
- Ensure DLT entity registration is complete before any outbound calling
- Implement AI disclosure in opening script from day one
- For your college use case, position as transactional/service calls with proper consent

documentation

- Do not use 140-series unless explicitly marketing; standard DIDs or 160-series (if eligible) are appropriate

3. Best STT Approach

Phase 1 (English): Deepgram Nova-3 or Flux

- Fastest streaming latency (<300ms)
- Best phone-audio WER among realtime APIs (21.8% on CallHome)
- Cheapest commercial option (\$0.0043/min)
- Built-in keyterm prompting for names/student IDs

Alternative: Google Cloud STT Chirp 2 if Indian accent recognition proves problematic with Deepgram.

Phase 2+ (Telugu): Google Cloud STT or fine-tuned Whisper. Deepgram's multilingual accuracy is significantly worse for Dravidian languages.

4. Best TTS Approach

Phase 1 (English): ElevenLabs v3 or Inworld TTS-2

- Highest naturalness and expressiveness
- Sub-200ms TTFB
- Good Indian English accent support

Budget alternative: Cartesia Sonic-3.5 (~40ms TTFB) if latency is absolute priority over expressiveness.

Phase 2+ (Telugu): This is the hardest gap. Options:

- VoisLabs for batch/pre-recorded Telugu content
- Google Cloud TTS for basic Telugu (robotic but functional)
- Custom fine-tuned model (significant research investment)
- Hybrid: English TTS for English words, Telugu TTS for Telugu words (complex orchestration)

5. Best LLM Strategy

Hybrid routing architecture:

- 90% of turns: Small fast model (Gemma 4 26B, Qwen3 8B, or Mistral Small)
 - Latency: <100ms TTFT
 - Cost: negligible
 - Handles: routine conversation, simple extraction, status updates
- 10% of turns: Escalate to GPT-4.1 or Claude Sonnet
 - Complex ambiguity resolution
 - Sentiment analysis
 - Multi-step reasoning
- Routing decision: Use a lightweight classifier or the small model's confidence score

Avoid: Using GPT-5.5 or Claude Opus for every turn. The cost is 50–100× higher with minimal quality benefit for routine conversation.

6. Best Real-Time Communication Architecture

DIY cascaded pipeline over managed platform for maximum quality control:
plain

Telephony (Exotel)

- WebSocket/RTP audio stream
- VAD + Endpoint Detection (semantic, neural)
- Streaming STT (Deepgram)
- Turn-taking orchestrator
- LLM Router (small → large)
- Streaming TTS (ElevenLabs/Cartesia)

→ Audio playback (minimal buffer)

Key architectural decisions:

- Use WebSockets for full-duplex audio streaming
- Run semantic VAD (not energy-based)
- Implement dedicated endpoint detection model
- Stream everything — no stage waits for previous to finish
- Maintain separate audio contexts for barge-in recovery
- Deploy in Mumbai or Bangalore (not US-East)

7. Best Backend Architecture

- Language: Python (FastAPI/Starlette) or Node.js — both have excellent WebSocket support
- Session state: Redis (hot state, sub-ms latency)
- Persistent storage: PostgreSQL (call records, transcripts) + ClickHouse or BigQuery (analytics)
- Queue: Redis Streams or RabbitMQ for async processing
- Orchestration: Custom orchestrator (not a managed platform) — you need control over turn-taking

8. Best Database

- Redis: Call session state, active WebSocket mappings, rate limiting
- PostgreSQL: User data, campaign configurations, call outcomes, transcripts
- Object storage (S3/R2): Call recordings, audio artifacts

9. Best Deployment Approach

- Cloud: AWS Mumbai (ap-south-1) or GCP Mumbai (asia-south1)
- Containerization: Docker + Kubernetes for horizontal scaling
- Edge: Run VAD/STT/endpoint models on edge nodes; LLM/TTS can be API calls
- Monitoring: Prometheus + Grafana for latency percentiles; custom dashboards for barge-in rate, endpoint accuracy, turn-taking latency

10. Best Development Roadmap

Table

Phas	Duratio	Goal	Key Deliverable
Phas	2–3 eks	Architecture dation	Single-call prototype with Exotel + Deepgram + T-4.1 + ElevenLabs

Phase a	4–6 weeks	English MVP	Working outbound campaign; basic interruption; structured extraction; transcript
Phase b	4–6 weeks	English quality	Semantic endpoint detection; barge-in <100ms; compressive TTS; <700ms E2E latency
Phase c	4–6 weeks	English scale	10+ concurrent calls; campaign dashboard; CSV export; basic analytics
Phase	8–12 weeks	Telugu research	Evaluate Telugu STT (Google, Whisper); evaluate Telugu TTS (VoisLabs, Google); build Tanglish test set
Phase	12–16 weeks	Telugu MVP	Basic Telugu conversation; code-switching handling; custom pronunciation
Phase	Ongoing	Multilingual	Expand to Hindi, Tamil based on Telugu learnings

Critical milestone before Phase 2: Your English system must achieve >90% structured extraction accuracy and <5% hang-up rate due to robotic feel. If you can't make English feel natural, Telugu will be impossible.

Final Challenge to Your Core Assumptions

- "We need a proper Indian number" → Correct, but SIM/eSIM is the wrong approach. Cloud telephony with a DID is the only scalable, legal approach.
- "English architecture will automatically produce Telugu" → Dangerously wrong. Telugu requires fundamentally different STT, TTS, endpoint detection, and LLM prompting. Budget 3–4× the English effort.
- "Open-source is cheaper" → Not necessarily true. When you factor in GPU costs, engineering time, and ongoing maintenance, commercial APIs are often cheaper for small teams.
- "We need a large LLM" → Wrong. A small model with smart routing outperforms a large model on latency, cost, and often user experience.
- "The hardest part is making the call" → Wrong. The hardest part is making the conversation feel natural. Telephony is a solved problem; turn-taking is not.
- "We can add compliance later" → Catastrophically wrong. TRAI's 2026 regulations can shut you down in 24 hours. Compliance must be in the MVP.

This project is technically feasible and commercially viable, but the path to genuinely natural conversation is steeper than most assume. Start with English, optimize for <700ms latency and flawless interruption handling, and treat Telugu as a Phase 2 research project with significant custom engineering requirements.

deepseek approach

Independent Research & Architecture Review: AI Voice Calling Platform for India

Date context: My training data runs through approximately mid-2025. I cannot browse the live web, and the current date in your environment is August 2026. Where facts may have shifted (pricing, specific Telugu model quality, regulatory changes), I explicitly flag uncertainty. Everything below distinguishes verified fact, provider claim, technical inference, and my recommendation.

TL;DR / Executive Summary

1. Do not build the telephony layer from scratch. The highest-leverage decision is to use an Indian CPaaS provider (Exotel or Ozonetel) for Indian DIDs and outbound calling, and either (a) use a managed AI-voice orchestration layer (Vapi, Retell, Sarvam AI) or (b) build your own orchestration on top of Pipecat/LiveKit with best-in-class pluggable STT/TTS. Building your own SIP media server is a distraction.
2. The quality differentiator is turn-taking and interruption handling, not the LLM. Most "robotic" AI callers fail because of poor VAD/endpointing and slow TTS, not because the LLM is slow. Invest engineering effort there first.
3. Indian number provisioning is the biggest regulatory/operational friction. Expect KYC, DLT registration, and consent documentation. An individual developer or student cannot easily get a production Indian DID. Partner with a registered entity or use a CPaaS that handles DLT on your behalf.
4. For English first: Use a streaming STT (Deepgram or Google), a fast LLM (Gemini 2.0 Flash or GPT-4o-mini class), and a streaming TTS (OpenAI TTS, Azure en-IN, or ElevenLabs if budget allows). Target total user-stops-speaking-to-AI-audio-starts under 700ms.
5. For Telugu later: Do not assume the same pipeline works. You will likely need Sarvam AI or [Gnani.ai](#) for STT/TTS because they are Telugu-first and handle code-switching. Realistic Telugu quality depends on these Indian vendors, and you should pilot them in Phase 2 before committing to a single architecture.
6. Cost per minute: Realistic ₹3–₹15 (USD \$0.04–\$0.18) depending on TTS choice and scale. This is cheaper than a human telecaller on a per-minute basis, but the comparison is misleading—AI quality still trails humans in difficult conversations.

7. Biggest risks: KYC/DLT friction, Google Dialer spam-flagging, Telugu TTS naturalness, and user experience of interruptions feeling "too sensitive" or "too stubborn."

1. Telephony Options in India

1.1 The Core Distinction: Who carries the call vs. who processes the audio

You need three things:

- Number (DID): an Indian phone number to call from.
- Call control API: ability to dial, track status, bridge audio.
- Real-time audio access: ability to get bidirectional raw audio (typically 8 kHz μ -law PCM for PSTN) into your AI pipeline via WebSocket, SIP/RTP, or proprietary media streams.

The providers below differ primarily in which of the three they actually give you in India.

1.2 Comparison of major approaches

Provider / Approach	Indian DIDs	Outbound India	Real-time audio to AI	KYC/DLT burden	Concurrency	Notes
Exotel (Indian CPaaS)	Yes	Yes, strong	WebSocket audio streaming available	Medium — entity KYC; DLT handled	High (hundreds)	Strong India play. Good API. Enterprise-focused.

			via "Exotel AI" / "Talk to AI"			
Ozonetel (Kookoo)	Yes	Yes	WebSock et streamin g available	Medium	High	Well-docu mented. More developer- friendly than Exotel historically
Kaleyra (Tata Communi cations)	Yes	Yes	Media streams available	Medium	Very high	Enterprise. Acquired by Tata in 2024. Strong on CPaaS but AI-orchest ration is less turnkey.

Twilio	Limited in India (inbound DIDs discontinued for new accounts ; outbound via regulator y bundle)	Yes, via partner carriers	Yes (Media Streams over WebSock et)	High for India numbers; easier via Twilio's global number pool, but those aren't Indian	High	Media Streams are the gold standard for AI integration s, but Indian DID acquisitio n is painful.
Plivo	Yes (Indian DID available)	Yes	Limited AI-specific media stream	Medium	Medium	Solid CPaaS, cheaper than Twilio. AI integration s less turnkey.
Sarvam AI (Voice Agent platform)	Partners with Indian telephon y	Yes	Fully managed voice pipeline	Medium (they handle much)	Depends on plan	Telugu-firs t AI voice agents. Strong for Indian

						languages . Less "bring-your- -own-mod el" flexibility in early versions.
Gnani.ai	Partners with carriers	Yes	Managed voice AI	Medium	High	Indian enterprise voice AI. Telugu STT/TTS strong.
SIP trunk providers (e.g., Tata, Airtel, enterprise SIP)	Yes	Yes	You must build media server (FreeSWI TCH)	High	Very high	Full control, but you own media routing, NAT, codecs, RTP. This is a multi-wee k

						engineering effort.
SIM/eSIM approach	Uses mobile phone number	Yes, via modem /gateway	Raw audio if you build the modem interface	Very low (just a SIM)	Very low (per SIM)	Not scalable. A SIM can realistically handle 1–3 concurrent calls with poor quality. Using a 4G/5G modem in a server is fragile.

1.3 SIM/eSIM vs. virtual numbers vs. SIP – what's actually appropriate?

Technical inference (strong): A normal SIM/eSIM in a phone or USB modem is not appropriate for a calling platform. Limitations:

- Mobile network operators often block or degrade concurrent voice calls on consumer SIMs. A standard SIM supports at most 2–4 simultaneous voice calls, and call quality over a relayed modem is poor.
- You'd be building your own GSM stack, which is a massive detour from the actual value (conversation quality).

- Caller ID would be a personal mobile number, which looks legitimate but doesn't scale and creates personal liability.

Recommendation: Use cloud telephony with Indian DIDs via a CPaaS. This gives you proper concurrency, call control, and regulatory compliance without the fragile hardware path.

1.4 The "AI-voice-agent platform" shortcut

Providers like Vapi, Retell AI, LiveKit, and Bolna abstract the entire telephony+AI pipeline. They each have different India coverage:

- Vapi: Works with Twilio/Telnyx/Exotel; Indian DID support depends on the underlying provider. Vapi handles WebSocket audio, VAD, STT/TTS plugin, LLM orchestration, and structured data extraction. Fast to start.
- Retell AI: Strong turn-detection model. Telephony via Twilio/Vonage/SIP. Indian DIDs possible via Plivo/Exotel connector.
- LiveKit: Open-source WebRTC/agents framework. You bring your own telephony via SIP; stronger for building custom pipelines.

Key inference: If your goal is to get a high-quality English demo fast, using a managed orchestrator (Vapi/Retell) with an Indian CPaaS for numbers is the pragmatic path. If you need deep control over turn-taking (which you said is a differentiator), you may eventually build your own orchestrator, but do not start there.

2. Real-Time Voice Architecture: Critical Components

2.1 The pipeline

text

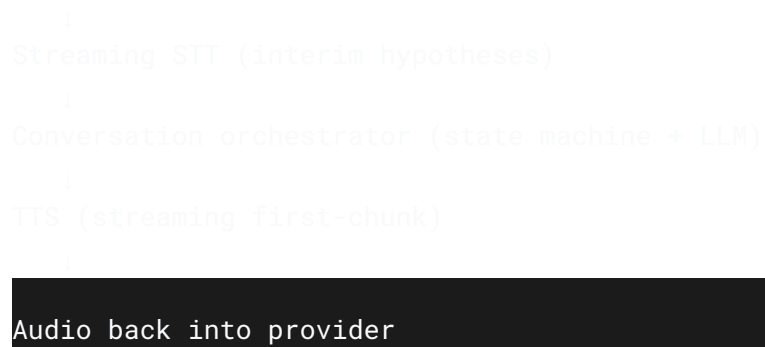
User speaks

↓

PSTN → Provider (SIP/RTP) → Audio stream (WebSocket/PCM)

↓

VAD / Endpoint detection



2.2 Component-by-component analysis

Component	Role	Impact on perceived quality	Notes
VAD (Voice Activity Detection)	Detect whether there is speech energy in the audio	Very high	Bad VAD causes either clipped speech or long silences. Use adaptive thresholds, not fixed energy.
Endpoint detection	Decide when the user has finished their turn	Extremely high	This is the #1 source of "robotic" feel. Pure silence threshold fails for hesitation. Need semantic-aware endpointing.

STT (Streaming)	Convert speech to text, emit interim results	High	Use interim results to start LLM pre-warming. Accuracy of Indian English matters.
Turn-taking model	Decide who speaks when	Extremely high	Should incorporate VAD, syntax, semantics, and user speaking style. Some vendors (Retell) sell this as a service.
Barge-in / interruption	Stop TTS when user starts speaking	Very high	Must stop within ~50~150ms of user speech onset. Full-duplex audio is mandatory.
LLM	Generate response, track state, extract entities	Medium (for quality)	A slow LLM adds latency; a wrong LLM fails extraction. But the LLM is rarely the quality bottleneck if chosen correctly.

TTS (Streaming)	Produce speech from text	Very high	First-audio latency matters most. Naturalness, pacing, and sentence endings matter for trust.
Session memory	Maintain context, slots, user intent	Medium	Use Redis for ephemeral state + Postgres for results. Keep context window tight.
Tool/function calling	Query knowledge base, validate data, update DB	Medium	Keep tools fast; avoid synchronous API calls that block the turn.
Call state machine	Track dialing, ringing, in-progress, completed	Medium	Finite-state machine, not ad-hoc code.

2.3 Largest perceived-quality levers (my inference)

1. Endpoint detection quality — decides whether the AI waits awkwardly or interrupts too early.
2. TTS first-audio latency and naturalness — the most audible difference.

- 3. Barge-in sensitivity – too sensitive = AI stops at coughs; too stubborn = feels like IVR.
- 4. STT accuracy on Indian English – mis-recognition cascades into wrong responses.
- 5. LLM speed – less important than people think if you pre-warm with interim STT.

3. Latency Targets and Pipeline Analysis

3.1 What humans perceive

- Human turn-taking gap in phone conversations: ~150–400ms.
- Below ~500ms total system response feels "good"; below ~700ms is acceptable; above ~1000ms feels robotic.

3.2 Complete latency chain (user stops speaking → AI audio starts)

Stage	Typical (naive)	Optimized	How to optimize
User stops speaking → VAD detects silence	150–300ms	50–150ms	Shorter silence threshold, adaptive VAD, or semantic endpointer
VAD → STT final hypothesis	200–500ms	100–300ms (interim)	Use interim hypotheses; don't wait for full final
STT → LLM prompt	0–50ms	0	Pre-assemble prompt

LLM → first response token	300–2000ms	200–500ms	Use fast model (Gemini Flash, GPT-4o-mini), streaming
LLM → TTS first audio	100–500ms	30–150ms	Streaming TTS; first-chunk synthesis
TTS audio → user's ear	20–100ms (PSTN jitter)	20–100ms	Provider-dependent
Total	770–3450ms	400–1200ms	

3.3 Where naive systems fail

Verified fact: The single biggest mistake is waiting for STT final result before calling the LLM. Final results are delayed by the trailing silence detector. High-quality systems use interim STT hypotheses to begin generating a response, then revise if the final hypothesis changes.

Technical inference: Another major source of latency is using a non-streaming TTS. Requesting the entire audio file from TTS before playback starts adds 500ms–2s. Streaming TTS with chunked synthesis is mandatory.

3.4 Practical target (my recommendation)

- English demo: Total user-stops-speaking → AI audio starts: 600–800ms.
- English production: 500–700ms.
- Telugu: Expect +100–200ms due to STT/TTS maturity, at least initially.

4. Interruption and Turn-Taking

4.1 Why this is the #1 hard problem

Turn-taking is about predicting intent, not just detecting sound. Humans use syntax, prosody, and semantics to know when the other person has yielded the floor. A purely energy-based VAD cannot do this.

4.2 The sub-problems

Problem	Naive approach	Better approach
User interrupting AI	Stop TTS when VAD detects user speech	Full-duplex audio; stop TTS within 50–150ms; do not stop on single-frame noise
User pauses while thinking	Treat as end of turn	Use longer silence threshold + semantic endpointer that checks if the utterance is a complete thought
Incomplete sentences	Respond immediately	Wait for syntactic completion or timeout; let LLM decide if the user trails off
Simultaneous speech	Just stop TTS	Consider both audio streams; if AI has almost finished a sentence, let it complete; if early in utterance, stop

Background speech / TV	Stop TTS and get confused	Use robust VAD + speaker separation; require sustained energy above threshold for N ms
False interruption (cough, uh)	Stop TTS	Require sustained user speech (e.g., 100ms) before interrupting; re-synthesize TTS from cut point
Long pauses	Timeout and prompt user	Adaptive timeouts; AI can say "Are you still there?" rather than restarting the whole turn
Agent deciding when to respond	Fixed silence threshold	Semantic turn-taking: use an LLM or dedicated model to judge if the user's utterance is complete

4.3 Architectural patterns used by high-quality systems

1. Full-duplex audio — The system listens while speaking. This is non-negotiable. (Twilio Media Streams, Exotel WebSocket, Pipecat, LiveKit all support this.)
2. Continuous VAD with hysteresis — A short "debounce" window prevents false triggers. Example: user speech must exceed threshold for 80–120ms before AI stops TTS.
3. Semantic endpointing — A small classifier (or the main LLM) decides "is the user done?" based on the content of the interim transcript, not just silence. Retell AI's

turn-detection model does this; you can also prompt the LLM: "Is this utterance complete? Answer only YES/NO."

- 4. TTS cut-point recovery – When interrupted, remember the last word boundary; resume from a natural sentence boundary rather than replaying the whole response.
- 5. Interrupt patience parameter – Different contexts need different sensitivity. A college absence call should be more patient than a sales call. Expose this as a tunable parameter.

4.4 My recommendation

Do not rely solely on a vendor's default turn detection. Even if you use Vapi/Retell, test with real Indian English audio containing hesitation, code-switching, and background noise and tune:

- Silence threshold (e.g., 400–600ms for Indian English)
- Interrupt activation time (e.g., 100–150ms of sustained user speech)
- Semantic endpointing on/off

This tuning is where your project will win or lose.

5. TTS Landscape (2024–2025 snapshot, flag for 2026 changes)

TTS	Natu	Str	First	English	Telugu	Cost	Verdict
	ralne	ea	-audi	Indian		(indica	
	ss	mi	o	accent		tive)	
		ng	laten				
			cy				

ElevenLabs	Very high	Yes	~200–400ms	Good en-IN voices	Yes, but quality uncertain for conversational Telugu	~\$0.30/1k chars (Turbo)	Best naturalness, highest cost. Use for English demo if budget allows.
OpenAI TTS (tts-1 / GPT-4o-mini-tts)	High	Yes	~100–300ms	Good	Telugu support, mixed quality	~\$15/1M chars	Best price/quality for English. Good first choice.
Azure Neural TTS	Good	Yes	~100–300ms	Excellent en-IN voices (Neerja, Arjun)	te-IN (Shruti, Mohan) decent	~\$16/1M chars	Solid, reliable, good Indian English. Slightly less "warm" than

							ElevenLabs/OpenAI.
Google Cloud TTS	Good	Yes	~100-300ms	Good en-IN	tel-IN available	~\$16/1M chars	Similar to Azure.
Cartesia (Sonic)	High	Yes	Very low (<150ms)	Good	Telugu growing	Tiered	Very low latency, strong for real-time. Check Telugu maturity in 2026.
Sarvam AI TTS	Good (Telugu-first)	Yes	Low	Good	Strong Telugu	Per-char, India pricing	Best Telugu option as of my knowledge. English is decent but not best-in-class.

Gnani.ai	Good	Yes	Low	Good	Strong	Enterprise	Solid
TTS	d	s			Telugu	rise pricing	Indian language option.
Open-source (VITS, XTTS v2, Kokoro, etc.)	Variable	Some	Variable	Variable	Poor	Compute cost	Not recommended for Telugu initially. XTTS v2 can be fine-tuned but engineering cost high.

5.1 Key inferences

- For English Phase 1: OpenAI TTS or Azure en-IN give the best cost/quality tradeoff. ElevenLabs is noticeably more natural but 20x more expensive per character. The difference matters when you scale.
- For Telugu Phase 2: The gap between open/global providers and Sarvam/Gnani is significant. Plan to switch TTS vendor for Telugu rather than assuming ElevenLabs/OpenAI will handle conversational Telugu well.
- Indian English accents: Azure en-IN voices are specifically trained on Indian English. OpenAI TTS handles Indian English well but is less India-specific. Test both with your actual scripts.

6. STT Landscape (2024–2025 snapshot)

STT	Streaming	Indian English	Telugu	Code-switching	Latency	Cost (indicative)
Deepgram (Nova-3)	Yes	Very good	Yes (but quality varies)	Some	Very low	~\$0.0059/min
Google Cloud STT	Yes	Good	Good (te-IN)	Some	Low	~\$0.016/min
Azure STT	Yes	Very good (en-IN)	Good (te-IN)	Some	Low	~\$0.015/min
Assembly AI (Universal)	Yes	Good	Yes	Moderate	Low	~\$0.0025/min
OpenAI Whisper (API)	Batch (chunking needed)	Very good	Good	Some	Higher	~\$0.006/min

Sarvam AI STT	Yes	Good	Best Telugu	Strong code-switching	Low	India pricing, competitive
Gnani.ai STT	Yes	Good	Strong Telugu	Strong	Low	Enterprise
Nvidia Canary (open)	Yes (with setup)	Good	Good	Moderate	Depends on GPU	Compute cost

6.1 Key inferences

- For English Phase 1: Deepgram or Google STT are the safe choices. Deepgram is cheaper and lower-latency; Google is very accurate on Indian English. Both stream well.
- For Telugu Phase 2: Sarvam AI or [Gnani.ai](#) are the realistic best options. Their models are trained on Indian telephone speech and handle code-switching better than the global providers. Pilot Sarvam in Phase 1 for English-to-Telugu comparison.
- Telephone audio (8kHz) degrades STT accuracy for all systems. Ask your telephony provider for 16kHz audio passthrough if possible. If not, choose an STT that explicitly trains on telephony audio (Deepgram, Google, Sarvam, Gnani).

7. LLM Strategy

7.1 Do you need a large reasoning model?

No. This is one of your assumptions I want to challenge.

A voice agent for structured calls (absence follow-up, lead qualification, appointment scheduling) is not a task that requires deep reasoning. The LLM needs to:

- Understand conversational intent from noisy STT output
- Decide what to say next from a constrained set of behaviors
- Extract structured entities (name, reason, return date)
- Stay within policy boundaries (don't promise things, don't hallucinate)

These are achievable with small, fast models.

7.2 Model comparison

Model class	Latency	Cost	Strengths	Weaknesses
Large reasoning (o1/o3, Sonnet 3.7+)	1–5s first token	High	Complex reasoning, long-horizon tasks	Massive latency, expensive, overkill for calls
Large general (GPT-4o, Claude Opus)	300–800ms	Medium-high	Good quality, strong tool use	Still overkill for most call flows
Small/fast (Gemini 2.0 Flash, GPT-4o-mini, Llama 3.1 8B on Groq)	100–500ms	Low	Low latency, cheap, sufficient for slot-filling and conversational control	May struggle with very complex user queries

Local (Llama 3.x 8B, Qwen 2.5 7B)	100–500ms (with GPU)	Very low marginal	Privacy, low cost at scale	DevOps burden, lower quality than cloud fast models
Deterministic logic	0ms	Zero	Perfect for validation, formatting, state transitions	Cannot handle open-ended natural language

7.3 Hybrid architecture (my recommendation)

text

Call flow:

1. Deterministic state machine handles dial, greeting, retry, hangup
2. LLM (small/fast) handles natural language:
 - Understand user utterance
 - Decide intent (e.g., "gives reason", "asks question", "wants to reschedule")
 - Extract entities (name, reason, date)
 - Generate the next response
3. Deterministic validators check the LLM's extraction:
 - Is the reason in an allowed enum? If not, ask LLM to re-classify
 - Is the date valid? If not, ask again
4. Structured output is stored via a separate, deterministic Pydantic schema

This reduces hallucination risk and speeds up extraction.

7.4 The hallucination challenge

Technical inference (high confidence): LLMs will hallucinate names, numbers, and commitments in a voice call unless you constrain them. Mitigations:

- Use JSON-schema / function-calling output rather than free text for structured data.
- Validate numbers against known formats (phone, dates).
- Keep a "ground truth" list of allowed entities in the system prompt (e.g., student name must match the CSV record).
- Use a separate small LLM call to verify extracted facts.

8. Cost Analysis

8.1 Per-minute cost breakdown (English, moderate quality)

Component	Cost basis	₹/min (approx)	\$/min
Telephony (Indian outbound, mobile)	₹0.30–₹1.00/min at moderate volume	₹0.30–₹1.00	\$0.004–\$0.012
Number rental	₹500–₹2000/month amortized	₹0.01–₹0.05	negligible
STT	Deepgram ~₹0.50/min; Google ~₹1.30/min	₹0.40–₹1.30	\$0.005–\$0.016
LLM	Gemini Flash / GPT-4o-mini: 2–5k in + 0.5–1k out tokens/min	₹0.20–₹1.00	\$0.002–\$0.012
TTS (OpenAI/Azure)	~₹1.30/1k chars; AI speaks ~400 chars/min	₹0.50–₹0.80	\$0.006–\$0.010

TTS (ElevenLabs)	~₹25/1k chars	₹10–₹15	\$0.12–\$0.18
Infra + storage + DB	Redis, Postgres, compute, S3 per call	₹0.10–₹0.50	\$0.001–\$0.006
Total (budget TTS)		₹1.51–₹4.65	\$0.018–\$0.056
Total (premium TTS)		₹11–₹18.65	\$0.13–\$0.22

8.2 Scale estimates

Assumptions: average call duration 2 minutes; average connect rate 50–60% for outbound.

Monthly calls	Minutes (connected)	Cost (budget)	Cost (premium)
100	~200	₹300–₹930	₹2,200–₹3,730
1,000	~2,000	₹3,020–₹9,300	₹22,000–₹37,300
10,000	~20,000	₹30,200–₹93,000	₹2.2L–₹3.7L
100,000	~200,000	₹3.0L–₹9.3L	₹22L–₹37L

8.3 vs. human calling

A human telecaller in India costs ₹15,000–₹25,000/month. At 160 working hours/month and 60% talk time, that's ~5,760 minutes → ₹2.6–₹4.3/min. But a human can handle only 1 call at a time, needs breaks, training, and supervision.

My inference: AI is cost-competitive at scale, especially with budget TTS. But the quality gap matters more than cost for your project's stated goal. Do not optimize for cost first; optimize for quality, then reduce cost.

9. Scalability

9.1 Concurrency bottlenecks

Scale	Bottleneck	Solutions
1 call	Development comfort	Manual testing
10 calls	Single-threaded orchestration, provider API rate limits	Horizontal orchestration workers, use async I/O (Python asyncio, Node.js)
100 calls	STT/TTS API rate limits; LLM concurrency; audio websocket management	Use streaming providers with high rate limits; provision more API keys; use connection pooling
1000 calls	Telephony provider concurrency caps; SIP trunk capacity; CPU for media processing	Multi-provider telephony; dedicated SIP trunk; use managed media servers (Asterisk/FreeSWITCH cluster)

9.2 Architecture for scale

- Stateless orchestration workers that consume a queue of "call sessions." Each active call is a WebSocket connection held by a worker.
- Redis for session state (transcript buffer, slot values, conversation history).
- PostgreSQL for final results, campaign state, exports.
- Queue (RabbitMQ/Kafka or Redis streams) for call commands: dial, hangup, retry.
- Media server only if you're doing SIP directly; if using CPaaS WebSocket, you skip this entirely.

Technical inference: For 1,000 concurrent calls, the cost of compute alone is manageable, but provider rate limits for STT/TTS/LLM will be the real constraint. You will need enterprise quotas with Deepgram/Google/OpenAI and possibly multiple API keys. Also, Indian CPaaS providers have per-account concurrency caps that may require negotiation.

10. Indian Telecom/Legal Considerations

10.1 KYC for Indian DIDs

Verified fact (as of 2025, likely unchanged 2026): To obtain a virtual Indian number (DID) for outbound calling, you must complete KYC. Requirements typically:

- Registered entity (company, LLP, partnership, sole proprietorship) – or individual KYC with some providers (harder)
- PAN card of entity/individual
- Address proof
- GST certificate if entity
- Authorized signatory documentation

Technical inference: An individual student developer will struggle. The pragmatic path is:

1. Partner with the college/institution as the KYC entity, or
2. Use a provider's sandbox/shared number for development (limited), or

3. Use a reseller that handles KYC on your behalf (many CPaaS resellers exist, but trust is an issue).

10.2 DLT (Distributed Ledger Technology) registration

Verified fact: TRAI's TCCCPR 2018 (amended) requires commercial communicators to register on DLT platforms (operated by Jio, Airtel, Vi, BSNL). This applies to SMS and voice commercial communications. Requirements:

- Entity registration on the DLT portals
- Header registration (for SMS) – for voice, this maps to caller ID registration
- Content template registration (for voice: call scripts/intent templates may need approval)
- Consent management: consent from recipients must be captured and logged
- Penalties for non-compliance, including number disconnection

Technical inference: The category matters:

- Transactional calls (related to an existing relationship, e.g., college notifying its own student about absence) have lighter consent requirements than promotional calls.
- A college calling its enrolled student about attendance is likely transactional, but the college should still have the student's consent (e.g., via admission forms) and maintain records.
- Some CPaaS providers (Exotel, Ozonetel) handle DLT registration as part of onboarding. Use them.

10.3 Recording and privacy

Verified fact: Indian law (IT Act, Indian Telegraph Act) requires that if you record calls, you must inform the other party and/or obtain consent. A standard practice is to play a short announcement: "This call is being recorded for quality and training purposes."

Technical inference (strong): You should do this for your prototype too. It reduces legal risk and sets user expectations. The DPDP Act 2023 (Digital Personal Data Protection Act) is in effect with rules being finalized; it requires consent for processing personal data, data minimization, and breach notification. This applies to storing transcripts, names, phone numbers, etc.

10.4 Automated calling restrictions

Verified fact (as of 2025): India does not have a blanket ban on automated outbound calls like the US TCPA. However:

- TRAI's DND registry allows users to opt out of promotional calls.
- Since 2024, Google Dialer on Android flags suspected spam numbers. Many shared cloud-telephony numbers get flagged, which destroys trust.
- TRAI has been increasingly concerned about AI voice calls; there were discussions in 2024–2025 about mandating disclosure of AI callers ("I am an AI assistant").
- In late 2024, Airtel and Jio launched their own AI voice services, so the direction is toward more AI calls, not fewer, but with disclosure expectations.

My recommendation: For the college absence use case:

1. Ensure the college has student consent (admission form).
2. Classify as transactional, not promotional.
3. Disclose AI nature at the start ("This is an AI assistant from [College] calling regarding your attendance.").
4. Play a recording notice.
5. Use a dedicated Indian DID with clean reputation, not a shared pool number.

11. English → Telugu Transition

11.1 What changes

Component	English Phase 1	Telugu Phase 2	What breaks
STT	Deepgram/Google en-IN works well	Sarvam/Gnani preferred	Global STT struggles with casual Telugu; code-switching is the killer

TTS	OpenAI/Azure/ElevenLabs	Sarvam/Gnani strongly preferred	Global TTS often has unnatural Telugu prosody; names and numbers garbled
LLM prompt	English system prompt	Telugu system prompt + transliteration policies	Hallucination risk higher; need Telugu-native constraint validation
Entity extraction	English NER works	Indian names/places/dates in Telugu are hard; use Telugu-specific validators	Date/time formats, student names in Telugu script
Turn-taking	Tuned for English prosody	Telugu has different pause patterns; retune VAD/endpointing	English tuning won't transfer
Code-switching	Minor	Major – Telugu speakers mix English words (Tanglish/Tenglish)	STT must handle both scripts in same utterance
Phone audio	8kHz degrades less	8kHz degrades Telugu more because of retroflex	Telugu recognition on PSTN is measurably worse

		and aspirated sounds	
Locale	Simple	Andhra vs Telangana	Regional variation in
handling		dialects, formal vs colloquial register	greetings, pronouns, verb endings

11.2 Biggest Telugu technical gaps (my inference, 2025 knowledge)

1. Conversational Telugu TTS — Global providers have Telugu, but it's often news-reading register, not conversational. Sarvam/Gnani are better but still maturing for "natural human phone conversation" quality.
2. Code-switching STT — Very few models handle "I am going to college, na? Actually, sir, nenu absent ayi...". Sarvam is the strongest I know of.
3. Telugu entity extraction — Extracting dates ("repu", "nellu", "varam"), names, and locations from colloquial Telugu requires Indian-specific training data.
4. Phone audio degradation — Telugu has retroflex consonants (ɻ, ʂ, ʣ, ʤ, ɳ) and aspirated stops that are poorly represented at 8kHz. Demand 16kHz audio from your telephony provider.

11.3 Phase 2 recommendation

Run a Telugu pilot early. Do not assume the English architecture transfers. In Phase 1 (English), already sign up for Sarvam AI's sandbox and test a small Telugu script. This will reveal:

- Whether your telephony provider can deliver 16kHz audio for Telugu
- Whether Sarvam's STT handles your target dialect
- Whether the TTS is acceptable to native Telugu speakers

Budget this pilot as a separate line item.

12. Competitive Landscape

12.1 Who's doing high-quality AI phone conversations (as of 2025)

Player	Focus	Strengths	Weaknesses
Retell AI	Developer platform, real-time AI voice agents	Strong turn-detection model, low latency, telephony integration, good dev UX	Less India-specific; Indian number support depends on underlying providers
Vapi	Developer platform	Large ecosystem, flexible "bring your own" STT/LLM/TTS, telephony handled	More configuration burden; India quality depends on providers
ElevenLabs Conversational AI	Voice agent platform	Top-tier TTS, brand, naturalness	Costly; less telephony depth in India; Telugu uncertain
OpenAI Realtime API	Voice-native API	Multimodal speech-to-speech, low-latency, high quality English	Expensive; telephony is DIY; Telugu uncertain;

			compliance burden on you
Google Gemini Live API	Voice-native API	Fast, good multilingual, competitive pricing	Telephony is DIY; developer docs evolving; Telugu quality unclear
Sarvam AI	Indian voice agents, Telugu-first	Best Telugu STT/TTS, India telephony partnerships, code-switching	Smaller ecosystem, less global tooling; English not as polished as global players
Gnani.ai	Indian enterprise voice AI	Strong Indian language models, enterprise trust, regulated industries	Less developer-friendly, more sales-led
LiveKit + Pipecat (open source)	Infrastructure	Maximum control, WebRTC/SIP, real-time agents	You assemble everything; no turnkey India telephony; engineering-heavy
Twilio + Voice Intelligence	CPaaS + AI add-ons	Mature telephony, Media Streams, developer trust	Indian DID friction; AI add-ons less flexible than

			dedicated platforms
Cisco/Genesys/AW S Connect	Enterprise contact center	Scale, compliance, integration	Not for a startup prototype; cost and complexity

12.2 What differentiates the strongest systems (my analysis)

1. Turn-detection model quality – Retell's biggest moat is their endpointer. This is where most DIY systems fail.
2. TTS naturalness + latency – ElevenLabs's voice quality is a real differentiator; Cartesia's low latency is too.
3. Indian language support – Sarvam and Gnani own Telugu for now. Global players are behind in code-switched Indian speech.
4. Telephony partnership depth – Exotel/Ozonetel have the Indian carrier relationships; a US-based platform (Vapi/Retell) without a local partner will struggle with Indian DIDs.
5. Developer flexibility – Vapi and LiveKit allow swapping STT/LLM/TTS; closed platforms (Sarvam, Gnani) don't, but deliver quicker for Indian languages.

12.3 My inference about the 2026 landscape

By mid-2026, expect:

- OpenAI Realtime API and Gemini Live API to have more telephony partners, but India DID support still gated by local KYC.
- Sarvam/Gnani expanding English quality, but still Telugu-first.
- More hybrid platforms (e.g., Vapi + Sarvam connectors) emerging.
- TRAI rules on AI call disclosure possibly formalized.

13. Critical Risks (Challenging Your Project)

13.1 Top risks ranked by severity

Risk	Likelihood	Impact	Mitigation
KYC/DLT friction prevents you from getting an Indian DID	High	Project-stopper	Partner with the college entity; use Exotel/Ozonetel who handle DLT; start this immediately
Google Dialer flags your number as spam	Medium-High	Trust destroyed	Dedicated DID, keep call volumes moderate per day, disclose AI identity, avoid repeated short calls
Robotic feel due to endpoint detection	High	Quality fails	Invest heavily in turn-taking tuning; consider Retell's endpointer or semantic endpointing
Telugu TTS quality is not human-natural	Medium-High	Phase 2 fails	Pilot Sarvam/Gnani early; budget for model fine-tuning or hybrid TTS

STT mis-recognizes Indian English on 8kHz telephone audio	Medium	Data extraction wrong	Use telephony-trained STT; request 16kHz audio; use deterministic validators
TTS cost blows budget at scale	Medium	Commercial failure	Start with OpenAI/Azure TTS; reserve ElevenLabs for demo only
LLM hallucination during information extraction	Medium	Wrong results, reputation damage	Use function-calling/JSON schema + deterministic validators; log and audit
Provider lock-in	Medium	Flexibility loss	Abstract STT/TTS/LLM behind interfaces; keep telephony provider swappable
Concurrency limits on STT/TTS/LLM APIs	Low-Medium	Scale ceiling	Provision multiple keys, negotiate enterprise quotas

Privacy/DPDP violation	Medium	Legal risk	Anonymize after export; store transcripts with consent; implement data minimization
Telephony audio degradation	Medium	Quality fails	Test with real PSTN calls; ask provider for 16kHz passthrough
User hangs up on AI	High (practical)	Campaign fails	Short, clear scripts; disclose AI; offer opt-out

13.2 Three assumptions I want to challenge

1. "Authentication and authorization are not required initially." — True for your web dashboard, but false for telephony. Indian number KYC/DLT is mandatory regardless of your dashboard. Do not defer this.
2. "English first, Telugu later is a smooth transition." — The English architecture will not automatically transfer. The gap between English and conversational Telugu in STT/TTS/entity-extraction is large. Treat Telugu as a separate project with its own pilot.
3. "We should build the whole pipeline ourselves for differentiation." — Your stated differentiator is conversation quality, not infrastructure. Building your own SIP/media server is weeks of work that does not improve conversation quality. Use proven platforms for telephony and audio transport; focus your engineering on turn-taking, prompt design, extraction validation, and language quality.

14. Recommended Technology Strategy

14.1 Guiding principles

1. Highest conversation quality per engineering hour.
2. Indian telephony credibility (DID, DLT, spam-free).
3. Switchable components (STT/TTS/LLM) to avoid lock-in.
4. Phase 2 Telugu readiness.

14.2 Recommendations

A. Telephony provider(s)

Primary: Exotel or Ozonetel (Indian CPaaS with WebSocket streaming and DLT assistance).

- Both offer Indian DIDs, outbound dialing, and real-time audio streaming suitable for AI integration.
- Exotel's "Exotel AI" suite is more mature for voice agents; Ozonetel is more developer-friendly.
- Choose based on: WebSocket audio format, pricing, concurrency, and whether they can provide 16kHz audio.

Alternative: Plivo (cheaper, Indian DID available) if you want a global CPaaS.

Avoid for initial phase: Twilio (Indian DID friction), SIP direct (engineering burden), SIM-based (unscalable).

B. Indian number approach

- Use a dedicated Indian mobile DID from your CPaaS.
- Complete KYC through the college or a registered entity.
- Ask the CPaaS to handle DLT registration for transactional calls.
- Disclose AI identity in the greeting.

C. STT

- English Phase 1: Deepgram (low cost, low latency) or Google STT (high accuracy for Indian English). Test both with your actual audio.
- Telugu Phase 2: Sarvam AI STT (Telugu-first, code-switching) or [Gnani.ai](#).

D. TTS

- English Phase 1: OpenAI TTS (tts-1 or gpt-4o-mini-tts) for cost/quality; Azure en-IN (Neerja/Arjun) if you prefer stronger Indian English; ElevenLabs only for investor demos.
- Telugu Phase 2: Sarvam AI TTS or [Gnani.ai](#) TTS. Do not rely on global providers for Telugu.

E. LLM strategy

- Primary: Gemini 2.0 Flash or GPT-4o-mini (fast, cheap, streaming, function-calling).
- Fallback: Groq-hosted Llama 3.x 70B for low-latency open-source option.
- Deterministic layer: Python/Pydantic validators for entity extraction, date parsing, and state transitions. The LLM generates natural language; validation is deterministic.

F. Real-time communication architecture

Option 1 (recommended for speed): Use Vapi or Retell AI as the orchestration layer. Configure your chosen STT/TTS/LLM. Connect to your Indian CPaaS for DIDs. Focus on prompt design and turn-tuning.

Option 2 (recommended for control): Build orchestration on Pipecat (Python) or LiveKit Agents. Connect to CPaaS WebSocket. Implement your own VAD/endpointing with semantic checks. This is more work but gives you full control over turn-taking and interruption.

Option 3 (not recommended initially): Build everything on FreeSWITCH + custom media server.

G. Backend architecture

- Language: Python (FastAPI) or TypeScript (Node.js). Python preferred for AI ecosystem.
- Orchestrator: Async service per active call (Pipecat actor model, or FastAPI + asyncio WebSockets).
- Queue: Redis Streams or RabbitMQ for call commands.
- Session state: Redis (with TTL) for live call context.
- Database: PostgreSQL for campaigns, contacts, results, transcripts.

- Object storage: S3-compatible (AWS S3 or MinIO) for audio recordings and full transcripts.
- Export: Generate Excel/CSV via backend job; use `openpyxl` or similar.

H. Best deployment approach

- Containerize services (Docker).
- Deploy orchestration workers on AWS/GCP/Azure (or an Indian cloud provider like E2E/Netmagic for data residency).
- Use managed Redis and Postgres for low ops overhead.
- For high concurrency, auto-scale workers horizontally.

I. Development roadmap

Phase 0 (2–4 weeks): Telephony setup.

- Choose CPaaS, complete KYC, get Indian DID, test outbound call.
- Set up WebSocket audio streaming and record a sample call.

Phase 1 (4–6 weeks): English MVP.

- Build orchestrator with Vapi/Retell or Pipecat.
- Integrate STT (Deepgram/Google), LLM (Gemini Flash/GPT-4o-mini), TTS (OpenAI/Azure).
- Tune turn-taking: silence threshold, interrupt sensitivity, semantic endpointing.
- Build campaign flow (CSV upload, call scheduling, result storage).
- Run a 100-call pilot with the college use case.
- Measure: latency, transcription accuracy, extraction accuracy, user experience.

Phase 2 (4–6 weeks): Telugu pilot.

- Swap STT/TTS to Sarvam/Gnani.
- Rewrite prompts and scripts in Telugu.
- Test with native Telugu speakers (both Andhra and Telangana dialects).
- Implement code-switching handling.
- Decide whether to keep unified architecture or fork language pipelines.

Phase 3 (ongoing): Productionization.

- Add authentication/authorization to the dashboard.
- Add DPDP compliance features (consent logs, data export, anonymization).

- Scale concurrency, negotiate enterprise API quotas.
- Build multi-campaign management, retry logic, reporting dashboard.

Summary of Where I Challenge Your Assumptions

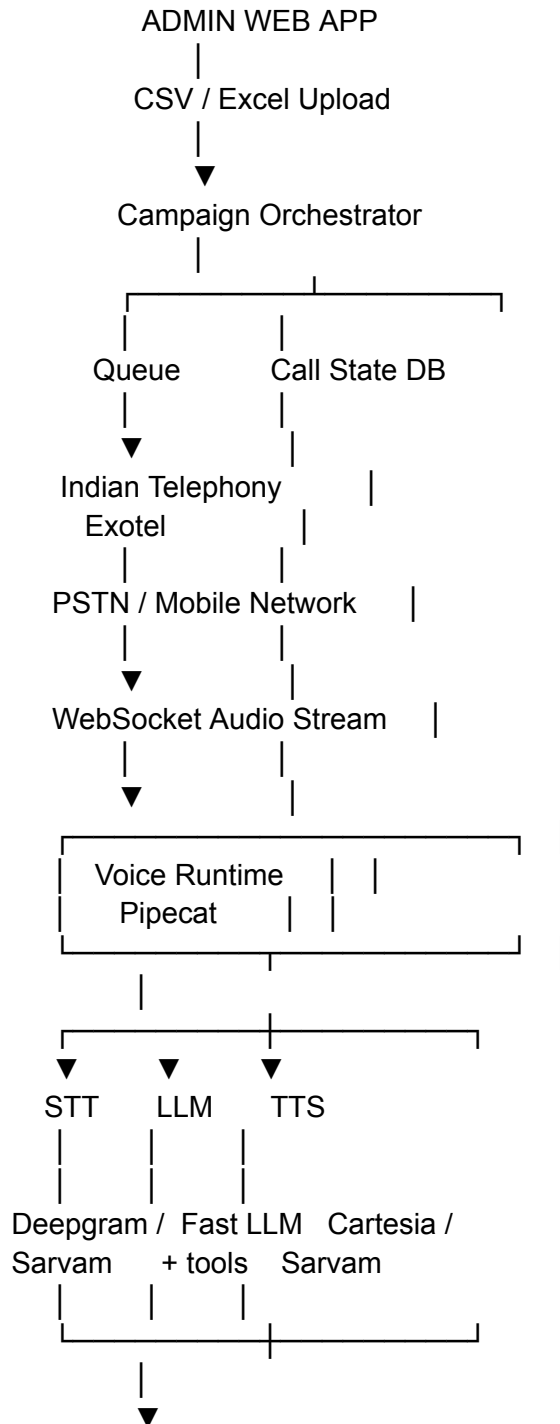
Your assumption	My challenge
"Authentication not required initially"	Telephony KYC/DLT is mandatory regardless. Do not defer it.
"English architecture will transfer to Telugu"	It will not. Telugu needs separate STT/TTS vendors and tuning. Plan a separate pilot.
"Building everything ourselves gives better differentiation"	Your differentiation is conversation quality, not infrastructure. Use proven platforms for telephony and audio transport; invest in turn-taking and language quality.
"Need a large LLM for intelligence"	No. Small/fast models with deterministic validation are better for structured calls.
"Open-source TTS/STT is cheaper/better"	For Telugu, open-source is worse. Indian vendors (Sarvam/Gnani) are the realistic best.
"SIM/eSIM could work"	Not scalable, not appropriate for concurrent calling.

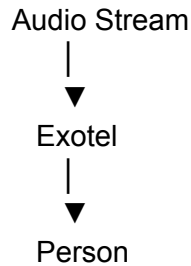
Final note: My information reflects the world as of approximately May 2025. Before you commit, verify current pricing, provider capabilities (especially Telugu STT/TTS quality and Indian DID availability), and Indian regulatory changes (TRAI AI voice disclosure rules may have evolved in 2026). The architecture and strategy above are designed to be robust to those changes—the key decisions (Indian CPaaS for telephony, managed orchestrator, pluggable STT/TTS/LLM, deterministic extraction layer) do not change if a specific vendor's pricing shifts.

GPT approach

1. Executive verdict

Recommended architecture





My first-choice stack

Layer	Recommendation	Why
Indian telephony	Exotel	Strong India positioning + real-time bidirectional streaming
Voice runtime	Pipecat	Open, modular, provider-independent
STT English	Deepgram Flux/Nova-3	Excellent real-time voice-agent tooling
STT India/Telugu	Sarvam Saaras	Much stronger India-language positioning
TTS English	Cartesia Sonic / ElevenLabs Flash	Excellent real-time quality
TTS Telugu	Sarvam Bulbul V3	India/Telugu focus
LLM	Fast model first , large model selectively	Voice does not need maximum reasoning everywhere
Database	PostgreSQL	Call/campaign/state data is relational
Queue	Redis + BullMQ initially	Simple campaign scheduling/concurrency control
Backend	Python/FastAPI for voice runtime + Node/Next.js if preferred for dashboard	Python ecosystem fits Pipecat
Deployment	Docker + AWS/GCP/Azure/Indian-region infrastructure where appropriate	predictable scaling
Storage	S3-compatible object storage	recordings/transcripts
Observability	OpenTelemetry + structured call events	voice debugging requires timelines

Exotel explicitly supports bidirectional WebSocket streaming for conversational bots, and its current documentation describes outbound calls, call status callbacks, recording, and direct WebSocket audio streaming.

Sarvam itself now documents an Exotel + Pipecat + Sarvam architecture for Indian-language voice agents, including Telugu, which is interesting because it validates the general architectural direction independently of our recommendation.

2. First assumption I would challenge

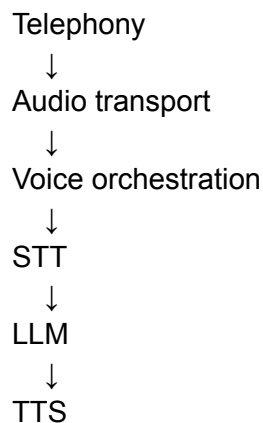
Your biggest potentially dangerous assumption is:

"We need an Indian virtual number, therefore we should find an Indian equivalent of Twilio."

Not quite.

The **telephony provider is not the voice AI platform**.

You should deliberately separate:



That separation is extremely important.

If you use a platform like Retell/Vapi for everything, you gain speed but lose control.

If you build everything yourself from raw SIP/RTP, you gain control but inherit a ridiculous amount of telecom engineering.

The sweet spot is:

Buy the telecom infrastructure. Build the intelligence layer.

3. Indian telephony landscape

Tier 1: Exotel

[Exotel](#)

Why I rank it #1 for this project

Exotel is unusually well aligned with what you need:

- Indian telephony
- Indian numbers
- outbound calls
- call status
- recording
- WebSocket streaming
- bidirectional audio
- Voicebot integration
- SIP
- enterprise telecom infrastructure

Its Voicebot Applet supports two-way audio over WebSocket, specifically for conversational bots.

It also supports an API flow where your application initiates an outbound call, Exotel connects the call, and the AI bot receives audio through WebSocket.

Important technical detail

Exotel's documented streaming format includes:

- 8 kHz mono PCM
- WebSocket
- JSON events
- media chunks
- DTMF
- call start/stop events

It also supports higher sample rates through its streaming configuration.

That is exactly the type of interface we want.

Weakness

Pricing isn't as transparently exposed as Plivo/Twilio.

You'll likely have to deal with:

- KYC
- business documentation
- account approval
- commercial negotiation
- concurrency/throughput discussions

For a serious production system, that's not necessarily bad.

For a random student wanting to call 500 people tomorrow, it is considerably less amusing.

4. Plivo

[Plivo](#)

Plivo is technically very interesting.

Current India pricing lists:

- outbound local calls: **₹0.60/min**
- local number: **₹250/month**
- browser/SIP: ₹0.34/min
- local number requirements include **Certificate of Incorporation + GST**.

More importantly, Plivo supports:

- bidirectional WebSocket streaming
- 8 kHz / 16 kHz PCM
- μ -law
- noise cancellation
- status callbacks
- audio playback back into the call
- interruption/clear-audio events.

That's excellent for a voice runtime.

But

The documentation's current India number requirements are a problem for a student prototype if you don't have an eligible business entity.

So:

Technically: excellent.

For your India-first prototype: probably #2 behind Exotel.

5. Twilio

[Twilio](#)

Twilio remains one of the strongest programmable telecom platforms globally.

But India changes the equation.

Twilio's current India guidelines explicitly state that outbound calls to Indian PSTN numbers can only be made from **international, non-Indian numbers**.

That directly conflicts with one of your requirements:

calls should appear legitimate/trustworthy with an Indian number.

Therefore:

Twilio is excellent globally.

Twilio is not my recommended primary Indian PSTN layer for this project.

Its India voice pricing currently lists approximately:

- \$0.0496/min to Indian mobiles
- \$0.0699/min to Indian local numbers

and it doesn't offer Indian voice-enabled numbers in the same way you need.

So don't build the architecture around Twilio merely because every tutorial on the internet does.

6. Knowlarity

[Knowlarity](#)

Knowlarity is a serious Indian cloud-telephony player.

Current published pricing includes:

- ₹1,999/month inbound contact center
- ₹2,999/month inbound + outbound contact center
- ₹1,499 to ₹1,999/agent/month for WebRTC offerings
- outbound pricing depending on product
- automatic outbound dialer/OBD offerings.

Problem

It is much more contact-center-oriented than developer-oriented.

For your project, I prefer a platform exposing clean real-time media primitives.

Verdict:

Good enterprise telephony option.

Not my first choice for building an experimental voice-AI runtime.

7. Ozonetel

[Ozonetel](#)

Worth considering for Indian production deployments.

It is especially relevant once you become:

- enterprise-facing
- contact-center-heavy
- campaign-heavy
- compliance-heavy

ElevenLabs currently offers an Ozonetel integration specifically for real-time AI agents handling Indian inbound/outbound calls.

That is a useful signal that Ozonetel is viable for AI voice infrastructure.

But again, for your **developer-controlled prototype**, Exotel is cleaner.

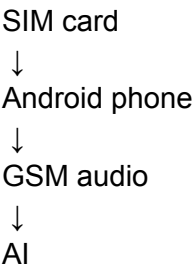
8. Indian telecom recommendation

My ranking:

Provider	India PSTN	Streamin g	AI friendliness	Developer control	Recommendation
Exotel	★★★★★	★★★★★ ★	★★★★★	★★★★★	#1
Plivo	★★★★★	★★★★★ ★	★★★★★	★★★★★	#2
Ozonetel	★★★★★	★★★★★	★★★★★	★★★	#3
Knowlarit y	★★★★★	★★★★	★★★★	★★★	#4
Twilio	★★ for your use case	★★★★★ ★	★★★★★	★★★★★	Avoid as primary India PSTN
SIM/eSIM	★★★	★	★	★	Don't use

9. Why SIM/eSIM is the wrong architecture

Don't build this around:



It sounds clever.

It is actually technical debt wearing a clever hat.

Problems:

- unreliable call control
- difficult concurrency
- limited automation
- carrier restrictions
- hardware dependency
- poor observability
- difficult scaling
- difficult number management
- difficult compliance
- no clean media API

For one experimental call, fine.

For a platform, absolutely not.

10. The telecom regulatory problem is bigger than you think

This deserves special attention.

TRAI's current framework explicitly addresses **auto-dialer and robo-calls**.

The 2025 regulatory changes distinguish:

- promotional voice calls
- service calls
- transactional calls
- auto-dialer/robocalls

and establish specific numbering approaches. TRAI's material says promotional auto-dialer/robocalls are associated with the **140 series**, while service/transactional voice calls are associated with **1600 or another allotted series**.

TRAI also defines a robocall as a call using artificial or prerecorded voice to interactively deliver a voice message without human involvement on the calling side.

This matters enormously.

Your college absence example is **not automatically equivalent to telemarketing**, but you cannot casually assume:

"It's educational, therefore regulations don't apply."

The actual classification depends on:

- sender
- purpose
- recipient relationship
- consent
- communication category
- provider implementation

And TRAI continues to update the framework. A 2026 consultation on further amendments was closed in April 2026.

Even more important

TRAI is now institutionalizing **AI/ML-based UCC detection** across operators.

So building something that behaves like:

10,000 calls

+

random 10-digit caller ID

+

automatic script

+

no consent

is an excellent way to discover that telecom operators have opinions.

Very strong opinions.

11. DLT and consent

TRAI's current guidance says commercial communications operate through registered headers and distinguishes promotional from service communications.

Therefore, before production outbound campaigns, your architecture needs a **compliance layer**, not just a dialer.

I would create:

Campaign

- |— purpose
- |— communication_type
- |— consent_source
- |— consent_timestamp
- |— recipient
- |— provider
- |— caller_id
- |— script_version
- |— recording_policy
- |— retention_policy

This becomes extremely important later.

12. Privacy and data protection

Your system potentially stores:

- phone number
- name
- student ID
- absence reason
- recordings
- transcripts
- summaries
- timestamps
- potentially sensitive personal information

India's DPDP Rules were formally notified in November 2025, with phased commencement.

The government's published material describes the framework as operationalized with an 18-month phased compliance timeline.

Therefore, even during the prototype phase, architect for:

- encryption
- deletion

- retention periods
- access controls
- audit logs
- consent records
- data minimization

Don't build a database containing five years of voice recordings because "storage is cheap."

Cheap storage has a remarkable ability to become expensive evidence.

13. Real-time voice architecture

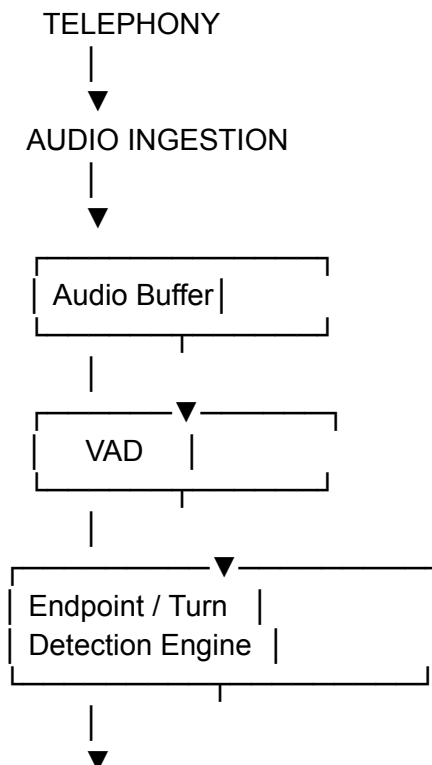
This is where your actual competitive advantage will come from.

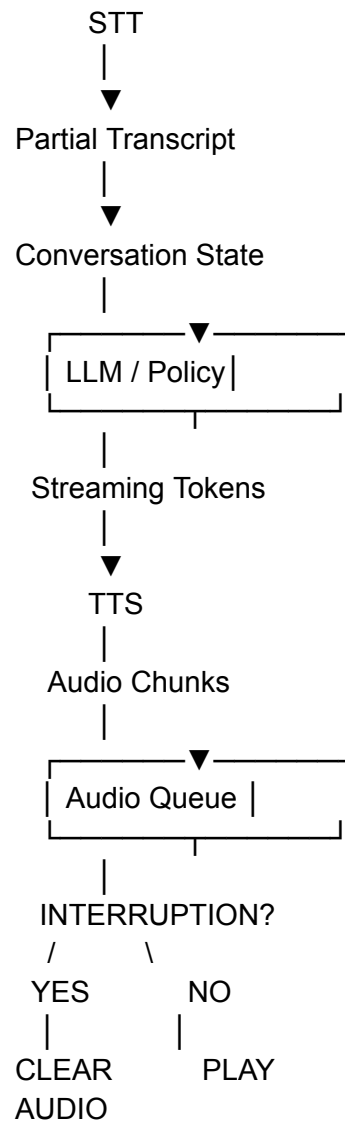
A good system isn't:

STT → LLM → TTS

That's the kindergarten diagram.

A serious architecture is:





This is much closer to what you actually need.

14. VAD is not endpoint detection

This distinction is critical.

VAD

Answers:

"Is someone speaking?"

Endpoint detection

Answers:

"Has this person finished their turn?"

Those are not the same problem.

Example:

"I was absent because... um... my..."

A naive VAD system may decide:

speech stopped

→ respond

Human conversation does not.

The person is still thinking.

You need a turn detector that understands:

- silence duration
- partial transcript
- syntax
- semantic incompleteness
- speech prosody
- conversation context

This is one reason Deepgram's current Flux model is interesting. It is specifically positioned for conversational speech recognition with turn detection and interruption handling.

15. Barge-in

This is probably the **single most visible quality feature**.

Suppose AI says:

"So your expected return date is..."

User:

"No, actually..."

The AI must stop.

Immediately.

Not:

"So your expected return date is... No, actually..."

That is the classic AI phone-bot embarrassment.

Your pipeline should have:

Incoming audio



VAD



Barge-in detector



CLEAR TTS BUFFER



STOP PLAYBACK



LISTEN

Plivo exposes explicit mechanisms for clearing buffered audio, and Exotel's voice-streaming architecture is designed for bidirectional conversational media.

16. Latency targets

Don't obsess over raw API latency.

Measure:

End of user's speech → first audible AI response

That's the metric the human experiences.

I'd establish these internal targets:

Metric

Target

Barge-in stop	<150 ms
STT first partial	<200 ms
End-of-turn detection	~200–400 ms
LLM first token	<200–400 ms
TTS first audio	<200–300 ms
First audible response	~400–800 ms
Excellent p95	<1.2 s
>2 sec	Feels noticeably robotic

These are engineering targets, not guarantees from vendors.

The biggest trick is **parallelism**.

Don't wait for:

complete STT
→ complete LLM
→ complete TTS

Instead:

partial STT
↓
LLM begins reasoning
↓
first useful phrase
↓
TTS begins
↓
audio starts

17. Streaming is not optional

TTS should produce:

"Yes, I understand..."

before the complete answer exists.

Then:

"Yes, I understand... you were absent because..."

continues streaming.

ElevenLabs documents WebSocket TTS specifically for real-time text input and says its Flash models have roughly 75 ms model inference latency under representative conditions, while warning that real end-to-end latency will be higher.

Sarvam's Bulbul V3 currently advertises sub-250 ms first-byte streaming and 8 kHz telephony optimization.

18. STT recommendation

English

#1 Deepgram

[Deepgram](#)

Deepgram's current Flux model is explicitly aimed at conversational voice agents and includes turn detection/interruption handling. Nova-3 is positioned for noisy, multilingual and crosstalk-heavy audio.

Current published prices are roughly:

- Nova-3 monolingual: ~\$0.0048/min
- Nova-3 multilingual: ~\$0.0058/min
- Flux English: ~\$0.0065/min
- Flux multilingual: ~\$0.0078/min.

That's extremely cheap relative to the overall voice stack.

#2 Sarvam

[Sarvam AI](#)

Sarvam is far more interesting once Indian languages enter the picture.

Current Saaras V3 supports 22 Indian languages + English and supports:

- real-time streaming
- code mixing
- Telugu
- Indian English
- word timestamps
- speaker diarization.

Its published API pricing is ₹30/hour, or roughly **₹0.50/minute**.

My strategy

Don't choose one STT permanently.

Build:

```
STTProvider
├── DeepgramSTT
├── SarvamSTT
└── GoogleSTT
```

Then benchmark them against **your actual calls**.

19. TTS recommendation

This is more nuanced.

English

Cartesia

[Cartesia](#)

Very strong candidate for low-latency English voice.

ElevenLabs

[ElevenLabs](#)

Still one of the strongest choices for expressive English.

Its Flash/Turbo API is currently priced around \$0.05/1,000 characters, with multilingual models around \$0.10/1,000 characters.

Its WebSocket streaming API is specifically designed for real-time generation.

20. Telugu changes the equation

This is where your roadmap is correct.

Don't assume:

Great English voice



same model



Great Telugu

That assumption is wrong.

Telugu needs:

- pronunciation
- Telugu phonetics
- regional variation
- names
- numbers
- locations
- code-switching
- Tanglish
- colloquial speech
- speech rhythm

Sarvam is currently one of the most strategically interesting options here because Bulbul V3 explicitly supports Telugu and Indian-language voice generation, with code-switching and telephony optimization.

Sarvam's current documentation also lists Telugu as a supported STT language and supports mixed-language recognition.

Therefore:

English:

Deepgram → fast LLM → Cartesia/ElevenLabs

Telugu:

Sarvam STT → fast LLM → Sarvam TTS

And the rest of the orchestration remains unchanged.

That's exactly why the provider abstraction matters.

21. LLM strategy

You do **not** need the largest reasoning model for every sentence.

A phone conversation has a very different computational profile from an agent doing software engineering.

Most turns are:

"Yes."

"Okay."

"I understand."

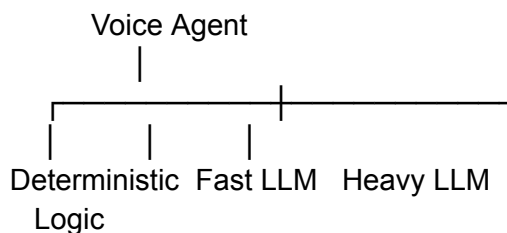
"What date?"

"Tuesday."

"Can you repeat that?"

Using an enormous reasoning model for that is like hiring a constitutional lawyer to decide whether you want tea.

Use three layers



Deterministic

Use code for:

- call state

- retries
- validation
- date parsing
- phone numbers
- campaign scheduling
- termination conditions
- structured extraction
- compliance
- tool permissions

Fast LLM

Use for:

- normal conversation
- paraphrasing
- intent recognition
- contextual responses
- natural follow-ups

Strong LLM

Use only when needed:

- ambiguous reasoning
 - complicated user requests
 - knowledge-intensive tasks
 - difficult tool planning
-

22. Native speech-to-speech vs cascaded pipeline

This is an important decision.

There are two architectures.

Cascaded

Audio

↓

STT

↓
LLM
↓
TTS
↓
Audio

Native realtime model

Audio
↓
Realtime multimodal model
↓
Audio

Google's Gemini Live API currently supports bidirectional real-time audio with native audio output over WebSockets.

Google's current Gemini pricing also exposes native-audio realtime models.

OpenAI also has a Realtime API for real-time audio interaction.

Native speech-to-speech advantage

Potentially:

- lower latency
- more natural turn-taking
- better prosody
- less pipeline overhead

Cascaded advantage

You can independently control:

- STT
- turn detection
- LLM
- TTS
- transcript
- structured extraction
- language
- provider
- debugging

For your project

I choose **cascaded first**.

Why?

Because your project isn't merely:

"Make a cool voice demo."

It's:

"Build an AI calling platform."

You need observability.

23. Pipecat is the right abstraction

[Pipecat](#)

Pipecat already supports real-time voice pipelines and WebSocket telephony transports.

Its architecture allows:

Telephony



Transport



VAD



STT



LLM



TTS

with interchangeable providers.

Its current quickstart uses:

- Deepgram
- OpenAI
- Cartesia

and reports sub-second full-round-trip behavior in its sample architecture.

More importantly, Sarvam has an existing Pipecat integration and Exotel example.

That combination is unusually attractive for your India → Telugu roadmap.

24. Why I don't recommend Vapi as the core

[Vapi](#)

Vapi is genuinely useful.

Current pricing is approximately:

\$0.05/minute platform fee

plus provider costs passed through at cost.

That's attractive.

But you're trying to learn/build the underlying architecture.

Vapi gives you:

telephony

+

STT

+

LLM

+

TTS

+

orchestration

conveniently.

The downside is that your actual differentiator becomes:

"We configured Vapi well."

That's not enough for the serious platform you're describing.

Use Vapi for:

- rapid experiments
- benchmarking
- reference implementation
- proving product-market fit

Don't make it your architectural foundation.

25. Retell is stronger, but same strategic problem

[Retell AI](#)

Retell is probably one of the best turnkey choices for high-quality voice agents.

Current pricing is approximately:

\$0.07–\$0.31/minute

depending on configuration. It includes a voice runtime, analytics, APIs, simulation, and 20 concurrent calls on the pay-as-you-go plan.

It supports a wide range of models and voices, including GPT, Claude, Gemini, Cartesia and ElevenLabs.

This is actually a very good product.

But:

You are trying to build the underlying product.

So I'd use Retell as a **benchmark**.

Build the same conversation on:

Retell

vs

your Pipecat stack

Then measure:

- response latency
- barge-in

- WER
- completion rate
- human rating
- cost

If your system can't beat it, you learn exactly where you're weak.

That's far more useful than blindly choosing one.

26. Competitive landscape

The serious ecosystem currently includes:

Infrastructure/platform

- [Exotel](#)
- [Plivo](#)
- [Twilio](#)
- [Ozonetel](#)
- [Knowlarity](#)

Voice AI platforms

- [Retell AI](#)
- [Vapi](#)
- [ElevenLabs](#)
- [LiveKit](#)
- [Pipecat](#)

India-focused AI

- [Sarvam AI](#)
- Haptik
- Yellow.ai
- Skit.ai
- Kore.ai
- Ozonetel

The strongest Indian enterprise players increasingly differentiate on **telephony reliability, compliance, multilingual support, and concurrency**, not merely the LLM. This is also reflected in current Indian enterprise voice-AI positioning from Exotel and Haptik.

27. Cost model

Let's use a realistic prototype assumption:

Average call

5 minutes

Then:

Calls	Minutes
100	500
1,000	5,000
10,000	50,000
100,000	500,000

Now approximate the AI layer.

STT

Sarvam:

₹30/hour

≈ **₹0.50/minute.**

Deepgram is dramatically cheaper on published raw STT pricing, but the correct choice depends on accuracy and Indian-language performance.

TTS

Sarvam Bulbul V3:

₹30/10,000 characters.

A five-minute conversation might produce around 600–1,000 assistant words depending on speaking behavior, so TTS cost can remain relatively low.

LLM

For a normal short phone conversation, the LLM is unlikely to dominate cost if you use an efficient model and aggressively manage context.

Infrastructure

At prototype scale:

Probably negligible compared with telecom + AI usage.

At 100k concurrent-scale minutes, infrastructure and observability become real costs.

28. Practical prototype cost estimate

For an India-first cascaded system, I would initially budget approximately:

₹1.5–₹3.5 per connected minute

for the AI + telecom stack, depending heavily on:

- telephony provider
- STT
- TTS
- LLM
- call duration
- recording
- infrastructure

This is an **engineering estimate**, not a vendor quote.

At 5-minute average calls:

Calls/month	Minutes	Approx operating cost
100	500	₹750–₹1,750
1,000	5,000	₹7,500–₹17,500
10,000	50,000	₹75k–₹1.75L

100,000 500,000 ₹7.5L–₹17.5L

Then optimize.

The raw numbers will change substantially once you negotiate Indian telecom pricing and choose exact model combinations.

29. Why your average call duration matters enormously

If you allow an AI agent to talk forever:

"Okay..."

"Right..."

"Yes..."

"So..."

"Let me understand..."

you burn money.

Your agent should have:

- objective
- state
- maximum duration
- maximum turns
- completion criteria
- early termination
- voicemail detection
- silence timeout

For the absence example:

1. Identify student
2. Confirm identity
3. Ask absence reason
4. Ask expected return
5. Ask whether support is required
6. Confirm
7. End

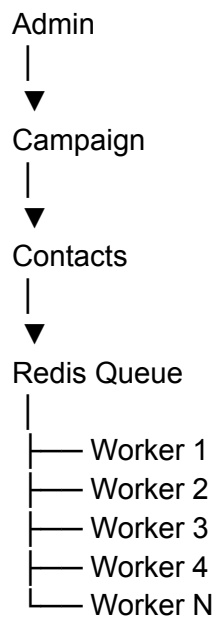
That's it.

Don't let the model discover philosophy halfway through the call.

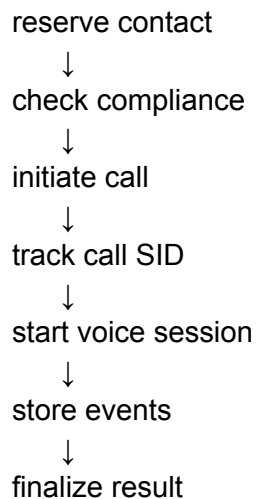
30. Campaign architecture

The campaign system should not directly call people from the web request.

Use a queue.



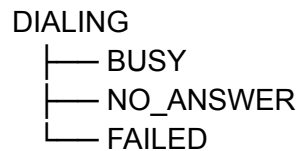
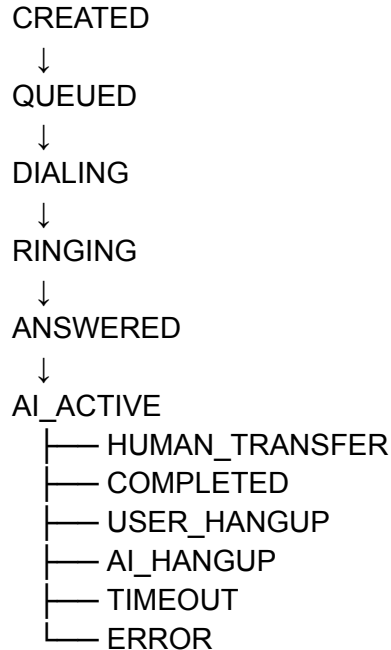
Each worker:



31. Call state machine

Don't let the LLM own call state.

Use deterministic state.



This gives you reliable reporting.

32. Structured extraction

Never depend on the final LLM summary to reconstruct everything.

Maintain structured state during the conversation.

Example:

```
{
  "student_id": "STU1024",
  "answered": true,
  "absence_reason": "fever",
  "expected_return_date": "2026-08-15",
  "needs_support": false,
  "confidence": 0.91,
  "call_duration_seconds": 182
}
```

The LLM can propose values.

Your application validates them.

For example:

LLM says:

"return date = next Friday"

↓

deterministic date resolver

↓

2026-08-21

This is far safer.

33. Don't let the LLM control everything

The architecture should be:

LLM = intelligence

Code = authority

The LLM can say:

"The student says they will return Monday."

Your code determines:

- what Monday means

- whether it is a valid date
- whether it falls within allowed range
- whether the field is complete

This principle becomes increasingly important in high-stakes voice systems.

Recent research into voice agents for Indian populations is also moving toward combining LLMs with deterministic validation and flow control rather than allowing the model to control the entire process.

34. Concurrency

Your system should have two separate concurrency limits.

Telephony concurrency

How many calls can Exotel place?

AI concurrency

How many voice sessions can your infrastructure process?

They are not the same.

At:

1 call

Nothing interesting happens.

10 calls

You discover bugs.

100 calls

You discover architecture.

1,000 calls

You discover telecom.

At 1,000 concurrent calls, you need:

- multiple workers
 - connection pools
 - queue partitioning
 - autoscaling
 - provider CPS management
 - Redis
 - backpressure
 - WebSocket lifecycle management
 - CPU/memory monitoring
 - model rate-limit management
 - provider failover
-

35. Don't scale the LLM blindly

Imagine:

1,000 calls

×

one LLM request every 5 seconds

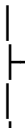
You suddenly have massive request throughput.

Instead:

1000 sessions



fast model



most requests

heavy model only when necessary

This dramatically reduces cost and rate-limit pressure.

36. Database

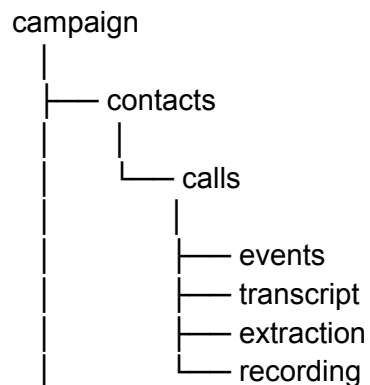
Use PostgreSQL.

Not MongoDB for this core data.

Your entities naturally look like:

organizations
campaigns
contacts
calls
call_events
conversation_turns
transcripts
extractions
recordings
providers
agents
agent_versions

Example:



This is relational data.

Postgres wins.

37. Frontend

Your initial dashboard can be:

Next.js
+
TypeScript
+

Tailwind
+
PostgreSQL

Pages:

/dashboard

/campaigns
/campaigns/:id

/campaigns/:id/contacts

/calls/:id

/analytics

/settings

Campaign page:

Campaign: Student Absence

Total: 1,000
Answered: 721
No Answer: 183
Busy: 61
Failed: 35

Avg duration: 2m 41s

Completion: 78.4%

Then export:

CSV
XLSX
JSON

38. The voice session object

Every active call needs an isolated session.

```
VoiceSession(  
    call_id,  
    campaign_id,  
    contact_id,  
    language,  
    state,  
    conversation_history,  
    extracted_fields,  
    audio_buffers,  
    provider_metadata  
)
```

Never put active call state into global process memory.

Because:

Call A
Call B
Call C

must never accidentally share:

"Okay, your fever..."

with another person.

That would be a spectacularly bad demo.

39. English → Telugu migration

The beautiful part is that most of your system doesn't change.

Doesn't change

Campaign engine
Call state
Database
Dashboard
Queue
Telephony
Analytics
Structured extraction
Tool system

Observability

Changes

STT

TTS

Language detection

Prompting

Pronunciation

Evaluation dataset

Turn-taking tuning

Normalization

Names

Numbers

So your architecture should have:

```
LanguageProfile(  
  language="en-IN",  
  stt_provider="deepgram",  
  tts_provider="cartesia",  
  normalization="english"  
)
```

and:

```
LanguageProfile(  
  language="te-IN",  
  stt_provider="sarvam",  
  tts_provider="sarvam",  
  normalization="telugu"  
)
```

Same agent.

Different speech stack.

40. Telugu should be tested separately

Build a Telugu evaluation dataset.

Not:

"It sounds good to me."

That's worthless.

Create ~500–2,000 representative utterances:

"నాకు జ్వరం వచ్చింది"

"నిన్న కాలేజీకి రాలేకపోయాను"

"రేపు వస్తాను"

"నాకు ఖచ్చితంగా తెలియదు"

"అది సోమవారం అనుకుంటా"

"ఫోన్ సరిగ్గా వినిపించడం లేదు"

Then add:

Code-switching

"నాకు fever ఉంది"

Tanglish

"Naku fever vachindi"

Names

"నా పేరు శ్రీనివాస్"

Numbers

"రెండు రోజులు"

"twenty second"

"22nd"

Regional variation

Telangana Telugu ≠ Andhra Telugu ≠ formal Telugu.

That's where your eventual moat could actually emerge.

41. Your real competitive advantage

It will **not** be:

"We use GPT."

Everyone uses GPT.

It will not be:

"We use AI."

Congratulations, the year is 2026.

Your moat could become:

Indian telephony

+

low-latency orchestration

+

excellent interruption handling

+

Indian accent STT

+

Telugu conversational data

+

structured workflows

+

reliability

+

evaluation infrastructure

Especially:

Your evaluation dataset.

If you collect thousands of real-world anonymized conversational examples and systematically measure:

- WER
- latency
- interruption success
- turn errors
- hallucination rate
- extraction accuracy

- task completion
- human naturalness rating

you start developing something genuinely difficult to copy.

42. Voice quality benchmark

Build a score out of 100.

Naturalness: 20

Does it sound human?

Latency: 20

Does it respond quickly?

Turn-taking: 15

Does it know when to speak?

Barge-in: 15

Does it stop immediately?

STT accuracy: 10

Does it understand correctly?

Task completion: 10

Did it accomplish the goal?

Extraction accuracy: 5

Did it record structured data correctly?

Recovery: 5

Can it recover from misunderstandings?

Then:

Human baseline

Retell

Vapi

Your system

Run the same conversations.

Now you're engineering instead of vibes-based benchmarking.

43. Biggest project risks

Here is where I'd be most worried.

Risk #1: Robotic turn-taking

Probability: very high

Most voice agents fail here before they fail at intelligence.

Risk #2: Latency

Probability: high

A brilliant model with 2-second response latency feels stupid.

Risk #3: Telecom restrictions

Probability: high

Especially outbound campaigns.

Risk #4: Number provisioning

Probability: medium-high

A student prototype may not have all documentation required for production numbers.

Risk #5: Telugu quality

Probability: high

English performance tells you almost nothing about Telugu conversational quality.

Risk #6: Provider lock-in

Probability: very high

If your business logic is embedded inside Retell/Vapi/etc., migration becomes painful.

Risk #7: LLM hallucinations

Probability: high

Especially when the agent is asked something outside its allowed scope.

Risk #8: Bad extraction

Probability: medium

The conversation may sound perfect while the database gets the wrong date.

That's worse than an obvious failure because it silently corrupts your results.

Risk #9: Cost explosion

Probability: medium

Especially if calls become long and model selection is careless.

44. My final provider strategy

Telephony

Primary

Exotel

Secondary

Plivo

Enterprise alternative

Ozonetel

Benchmark/global

Twilio

45. STT

English

Deepgram Flux / Nova-3

Indian English / Telugu

Sarvam Saaras

Backup

Google Speech / another provider

Don't hard-code one vendor.

46. TTS

English

Test:

1. Cartesia
2. ElevenLabs Flash
3. OpenAI
4. Google

Telugu

Start with:

Sarvam Bulbul V3

Sarvam currently advertises 35+ voices across 11 Indian languages, including Telugu, with streaming and telephony-oriented output.

47. LLM

Default

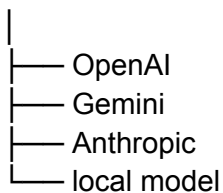
Fast model.

Heavy reasoning

Only when required.

Architecture

LLM interface



Don't let the LLM provider become your platform.

48. Voice orchestration

Pipecat

This is my strongest architectural recommendation.

Not because it is trendy.

Because it gives you the thing you actually need:

control over the pipeline without having to implement the entire media-processing stack from scratch.

Pipecat explicitly supports WebSocket telephony transports and provider components.

49. Alternative architecture

There is another viable route:

Exotel

↓

ElevenLabs Agents

↓

LLM + TTS + STT + orchestration

ElevenLabs now has direct Exotel integration using WebSocket and positions it specifically for Indian contact centers.

This could give you a much faster proof of concept.

But I would use it as:

Benchmark / rapid MVP

rather than:

core architecture

because you want control over the speech stack and Telugu migration.

50. Recommended development roadmap

Phase 0: Telephony proof

1–3 days

Goal:

Browser/API



Exotel



Indian phone

Verify:

- number
- KYC
- outbound call
- caller ID
- call status
- WebSocket
- audio

Do not touch AI yet.

Phase 1: Audio pipeline

3–5 days

Build:

Exotel



WebSocket



Pipecat

↓
VAD
↓
STT
↓
TTS
↓
Exotel

Goal:

caller speaks → AI repeats/answers

Phase 2: Real conversation

5–10 days

Add:

- LLM
 - streaming
 - context
 - interruption
 - endpoint detection
 - silence handling
-

Phase 3: Structured workflow

3–7 days

Add:

Campaign
↓
Contact
↓
Call
↓
Extraction
↓

Result

Phase 4: Dashboard

5–10 days

Build:

- upload
 - contact preview
 - campaign start
 - live status
 - results
 - transcript
 - summary
 - export
-

Phase 5: Evaluation

1–2 weeks

Create:

- latency benchmarks
- WER benchmarks
- interruption tests
- extraction tests
- human rating
- failure dataset

This phase is more important than adding another 15 features.

Phase 6: Telugu

Only after English is genuinely good.

Then:

Sarvam STT

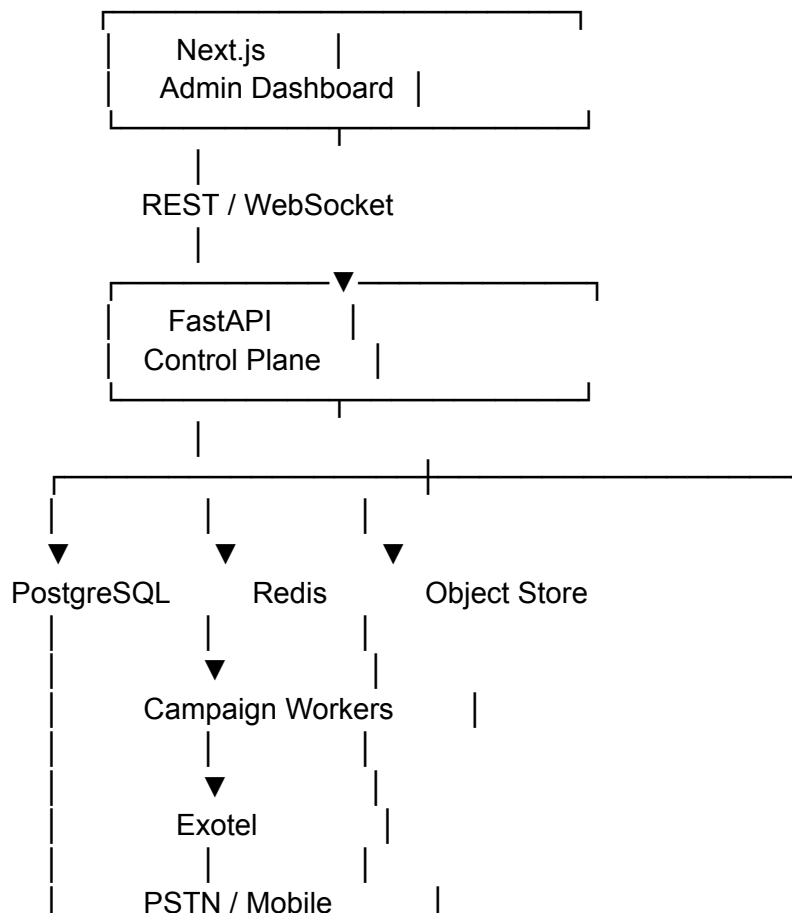
+
Sarvam TTS
+
same runtime

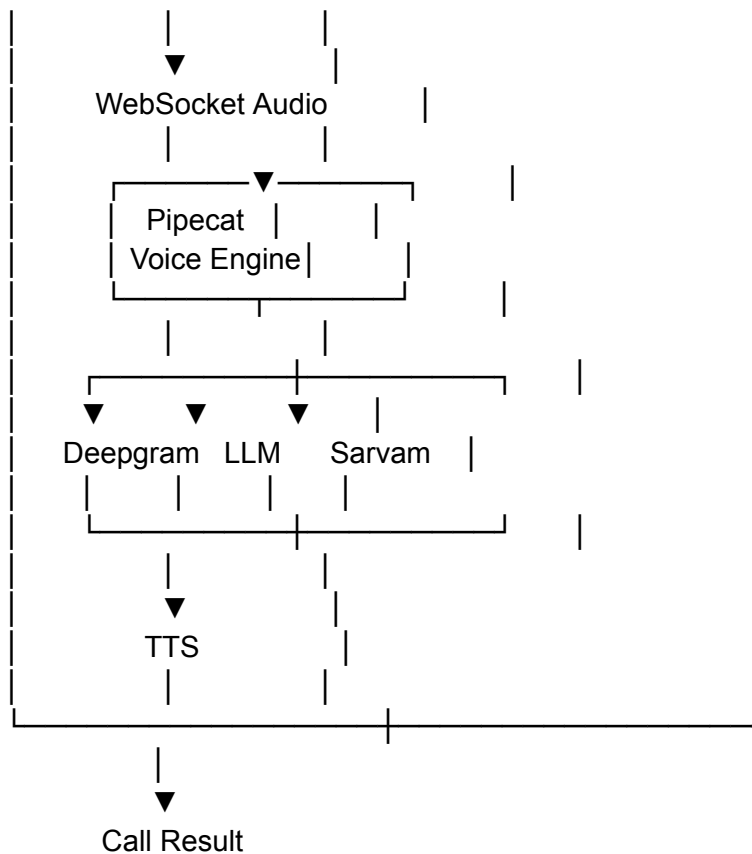
Add:

- Telugu
- Tanglish
- code-switching
- pronunciation dictionary
- regional dialect tests

Sarvam's current speech stack already exposes Telugu, code-mixing, streaming and pronunciation-related capabilities, making it a strong candidate for this phase.

51. The architecture I would actually build





52. One important change to your original plan

You described the system as domain-independent from the beginning.

I would **not** do that.

Build the runtime generically.

But build the **first agent around one narrow workflow**.

For example:

College absence verification

Why?

Because voice quality is difficult enough.

You don't need:

absence
sales
customer support
appointments
collections
surveys
lead qualification

simultaneously.

The platform should be generic.

The first agent should not be.

53. The first MVP should therefore be

"AI Student Absence Calling Agent"

Input:

student_id,name,phone
ST001,Rahul,+91...
ST002,Priya,+91...

Agent:

Hello Rahul, this is an automated call from XYZ College.

We noticed that you were absent today.

Could you tell me the reason?

Then:

Reason?
Expected return?
Need assistance?

Then:

Thank you. We have recorded that information.

Have a good day.

Output:

student_id
name
phone
call_status
absence_reason
expected_return
support_required
duration
confidence
transcript
summary

That gives you a complete vertical slice of the entire platform.

54. Final stack decision

If we were starting development **today**, my stack would be:

Frontend

Next.js + TypeScript + Tailwind

Control API

FastAPI

Voice Runtime

Pipecat

Telephony

Exotel

Queue

Redis + BullMQ / Celery

Database

PostgreSQL

Storage

S3-compatible

English STT
Deepgram Flux

English TTS
Cartesia / ElevenLabs Flash

Indian/Telugu STT
Sarvam Saaras

Telugu TTS
Sarvam Bulbul V3

LLM
Fast GPT/Gemini model
+
strong model fallback

Observability
OpenTelemetry

Deployment
Docker
+
AWS/GCP/Azure

CI/CD
GitHub Actions

The most important architectural rule:

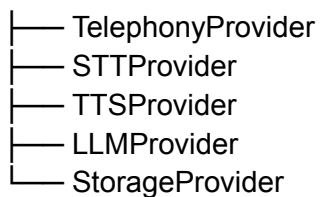
No provider-specific business logic inside the agent.

Instead:

VoiceRuntime



Provider Interfaces



That one decision will save you enormous pain when you move from English to Telugu.

55. Bottom line

Your original instinct to prioritize **conversation quality over simply making calls** is correct. The architecture should therefore optimize the things humans actually notice: response latency, turn-taking, barge-in, speech recognition, pacing and recovery.

But I would change the implementation philosophy:

Don't build:

"A Vapi/Retell clone."

Build:

An India-first voice runtime that happens to use Exotel for telecom.

The strongest initial combination is:

Exotel + Pipecat + Deepgram + fast LLM + Cartesia/ElevenLabs

and then:

Exotel + Pipecat + Sarvam Saaras + fast LLM + Sarvam Bulbul V3

for Telugu.

The biggest technical moat is not the LLM. It is the **real-time orchestration + evaluation + Indian-language speech quality + telecom reliability**.

And the biggest non-technical risk is not model quality. It is **Indian outbound-call compliance and number provisioning**. TRAI's current framework explicitly regulates auto-dialer/robocall behavior, commercial communication categories and numbering, while its 2026 work is increasing automated UCC detection.

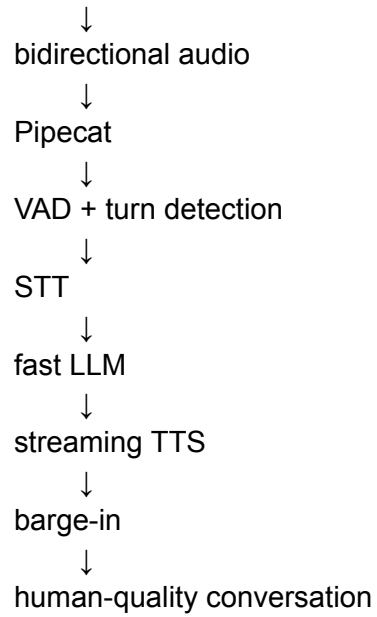
So I would **not start by building the dashboard**.

I would start with the hardest proof:

Exotel Indian number



real mobile call



If that loop feels genuinely good, everything else is engineering.

If that loop feels robotic, the dashboard is just a beautifully designed way to display failure.

gemini 3.6 flash

Critical Assumption Audit & Technical Reality Check

Before building a voice AI platform for the Indian telecommunications landscape, we must address three flawed architectural assumptions commonly held by engineering teams:

Assumption 1: SIM Cards or Mobile GSM Gateways Can Scale Outbound AI Calling

- The Flaw: Utilizing physical SIM cards, eSIMs, or local GSM gateways (such as GoIP boxes) to route AI calls is unsuitable for production systems.
-
- The Reality: Commercial SIMs operate under strict Indian TRAI/DoT Fair Usage Policies (FUP). Automated calling over regular consumer SIMs triggers instant network-level fraud detection, leading to IMEI/SIM blocking and phone number blacklisting. Furthermore, local SIM gateways offer zero native WebSockets, poor audio quality (lacking direct digital PCM capture), and no scalable concurrency. You must use enterprise-grade Cloud Telephony or Programmable SIP Trunking.
-

Assumption 2: An English Voice-AI Pipeline Can Effortlessly Adapt to Indian Languages

- The Flaw: Assuming that swapping an English STT/TTS engine with a Telugu STT/TTS engine on an existing pipeline will retain fluid, low-latency conversation quality.
-
- The Reality: Standard global models (OpenAI, Deepgram, ElevenLabs) degrade significantly on Indian telephonic audio (8kHz bandpass filtered), Indian proper nouns, regional dialects, and code-switching (Tanglish / Telugu-English). Indic languages require specific acoustic modeling, custom VAD tuning for longer conversational pauses, higher token density handling in LLMs, and localized TTS engines (e.g., Sarvam AI).
-

Assumption 3: Turn-Based AI Architecture ("Listen $\rightarrow$ Process $\rightarrow$ Speak") can feel Human

- The Flaw: Relying on standard sequential API calls where the system waits for audio to complete before processing.
-
- The Reality: Human phone conversations operate on continuous, full-duplex feedback. A user says "uh-huh" or pauses to think. Sequential pipelines cause high perceived latency ($>1.5\text{s}$) or aggressive, unnatural interruptions. Achieving human conversation

requires a full-duplex, streaming-first pipeline using bidirectional WebSockets or WebRTC/SIP, continuous energy/semantic VAD, and real-time audio frame manipulation.

•

1. Telephony & Indian Telecom Options

To make legitimate outbound calls in India with proper Caller ID, low latency, and continuous real-time audio access, your system architecture must integrate directly with Indian Telecom Service Providers (TSPs) or compliant international CPaaS platforms.

Architecture Options Comparison

Option	Inbound/Outbound Audio Path	Indian CLI Support	Regulatory Compliance	Concurrency Capabilities	Real-Time Audio Streaming
Indian Cloud Telephony (Exotel, Tata Tele, MyOperator)	HTTP Webhooks / WebSockets	High (Fixed Local / 140 / 160 Series)	High (Full TRAI/DLT Compliance)	High (Scalable via virtual channels)	Excellent (Exotel Voicebot Applet / Audio Streaming over WSS)
Global CPaaS with India POP (Plivo, Twilio India)	SIP Trunking / Bidirectional WSS	High (Must complete Indian Entity KYC)	High (Enforces TRAI DLT scrubbing)	Unlimited (Elastic SIP / Channel scaling)	Excellent (Direct WSS / Native LiveKit & Pipecat adapters)
Direct SIP Trunking (Tata Comms, Airtel Enterprise)	Pure SIP/RTP Protocol	Enterprise Direct CLI	Enterprise Legal Agreement Required	Very High (Dedicated Bandwidth E1/PRI/IP-SIP)	Raw RTP Audio (Requires custom SBC/RTP Media Gateway)

GSM Gateways / SIM Boxes	Analog/GSM to SIP	Single Mobile Number	Illegal / High Risk (Banned by TRAI for commercial transit)	Very Low (Limited by hardware ports & SIM locks)	Poor (A/D conversion distortion, high jitter)
--------------------------	-------------------	----------------------	---	--	---

Regulatory, KYC, and DLT Requirements in India

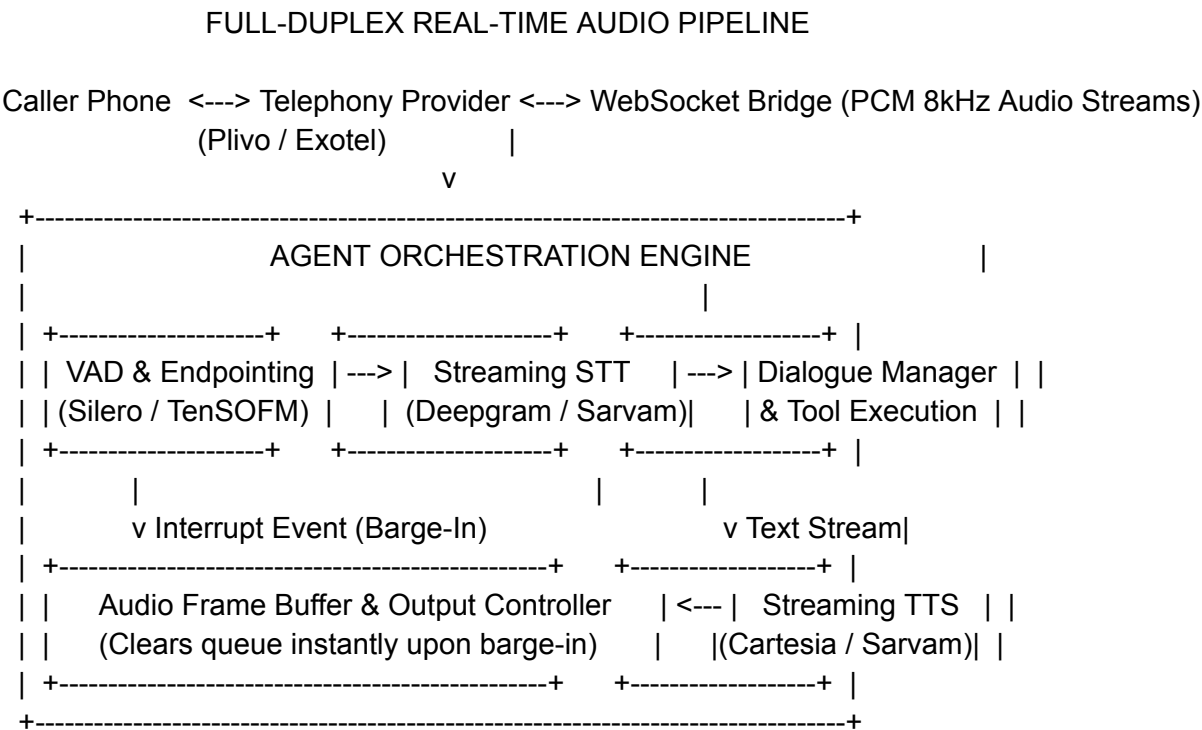
1. DLT (Distributed Ledger Technology) Registration: Every business placing commercial or automated voice calls in India must register as a Principal Entity on a telecom DLT portal (Jio, Airtel, Vodafone Idea, or Tata).
- 2.
3. 140 vs. 160 Number Series (TRAI Regulations):
4.
 - 140 Series: Mandatory prefix (+91-140-XXXXXXX) for Promotional/Telemarketing calls. Bypasses non-DND (Do Not Disturb) numbers only; blocked on DND numbers.
 -
 - 160 Series: Designated prefix (+91-160-XXXXXXX or 1600) reserved exclusively for Service and Transactional calls (e.g., student attendance alerts, BFSI reminders, OTPs). Calls can reach DND-registered numbers provided explicit prior consent exists.
 -
5. KYC Documents Required for Provisioning: Certificate of Incorporation, GST Registration, Company PAN, Authorized Signatory LOA (Letter of Authorization), and proof of physical office in India.
- 6.

Telecom Recommendation for Prototype vs. Production

- For Prototype: Use Plivo or Exotel. Both offer compliant Indian virtual numbers with direct Bidirectional WebSocket Audio Streaming (PCM 8kHz mono, Slinc/Base64). This bypasses the complexity of building custom Session Border Controllers (SBC) while keeping latency low ($<100\text{ms}$ telephonic transport).
-
- For High-Scale Production: Transition to Direct Enterprise Cloud SIP Trunking (Plivo / Tata Communications) connected directly to a self-hosted media server cluster (LiveKit SIP ingress/egress).
-

2. Real-Time Voice Pipeline Architecture

To achieve a natural human-like voice conversation, you must avoid coarse turn-taking. The diagram below illustrates the continuous, event-driven streaming pipeline:



Key Architectural Components

1. Telephony Bridge & Audio Transcoder: Converts telephony audio streams (G.711 μ -law/A-law, 8kHz PCM) into standard WebSockets PCM chunks (typically 20ms–100ms frames).
- 2.
3. Dual-Layer Voice Activity Detection (VAD):
 - Local Energy/Acoustic VAD: High-speed (e.g., Silero VAD, running in $<10\text{ms}$) running directly on incoming audio frames to detect instant vocal onset or interruption.
 -
 - Semantic/LLM VAD: Secondary analysis to determine if the user has reached a pause (thinking) versus a completed utterance (turn boundary).
 -
5. Streaming Speech-to-Text (STT): Maintains an open WebSocket connection, converting incoming chunks into interim transcriptions with sub-word timestamps and final transcriptions.

- 6.
- 7. Conversation Orchestration State Machine: Manages state transitions, session variables (student data, call status), tool execution, and prompt injection.
- 8.
- 9. Ultra-Low TTFT (Time-To-First-Token) LLM: Receives text context and streams output tokens immediately.
- 10.
- 11. Streaming Text-to-Speech (TTS): Processes token chunks, generates audio bytes (24kHz or 16kHz), and downsamples them to telephonic 8kHz PCM.
- 12.
- 13. Interruption-Aware Audio Playback Buffer: Holds audio frames being sent back over the WebSocket. If the VAD flags a barge-in event, it fires a clear_buffer signal to instantly purge unplayed audio.
- 14.

3. Latency Budget & Optimization Strategies

Human speech turn-taking typically occurs within a 200ms to 500ms window. In telephonic voice AI, a total round-trip response time exceeding 800ms creates an unnatural "walkie-talkie" cadence.

System Latency Budget

$$T_{\text{Total}} = T_{\text{VAD}} + T_{\text{STT}} + T_{\text{LLM_TTFT}} + T_{\text{TTS_TTFB}} + T_{\text{Network_Jitter}}$$

Component	Target Latency (Optimal)	Realistic Latency (Standard Setup)	Optimization Strategy
VAD / Silence Detection	100ms - 200ms	500ms	Use sub-frame acoustic VAD + speculative execution instead of waiting for long trailing silence.
Streaming STT	100ms - 150ms	300ms	Use WebSocket streaming models (e.g., Deepgram Nova-2) with interim transcription enabled.

LLM Processing (TTFT)	\$150\text{ms} - 250\text{ms}\$	\$600\text{ms}\$	Deploy fast inference engines (Groq, vLLM) using small, tuned instruction models (Llama-3.1-8B, Claude 3.5 Haiku, Gemini 2.0 Flash).
TTS Processing (TTFB)	\$100\text{ms} - 180\text{ms}\$	\$400\text{ms}\$	Stream audio on sentence fragments or first 3–5 tokens (e.g., Cartesia Sonic, ElevenLabs Turbo v2.5).
Network & Telephony Buffer	\$50\text{ms} - 80\text{ms}\$	\$150\text{ms}\$	Pin application deployment region to target market (e.g., AWS ap-south-1 Mumbai).
Total Round-Trip Latency	\$500\text{ms} - 860\text{ms}\$	\$1950\text{ms}+\$	Sub-800ms feels conversational; $>1.5\text{s}$ breaks natural flow.

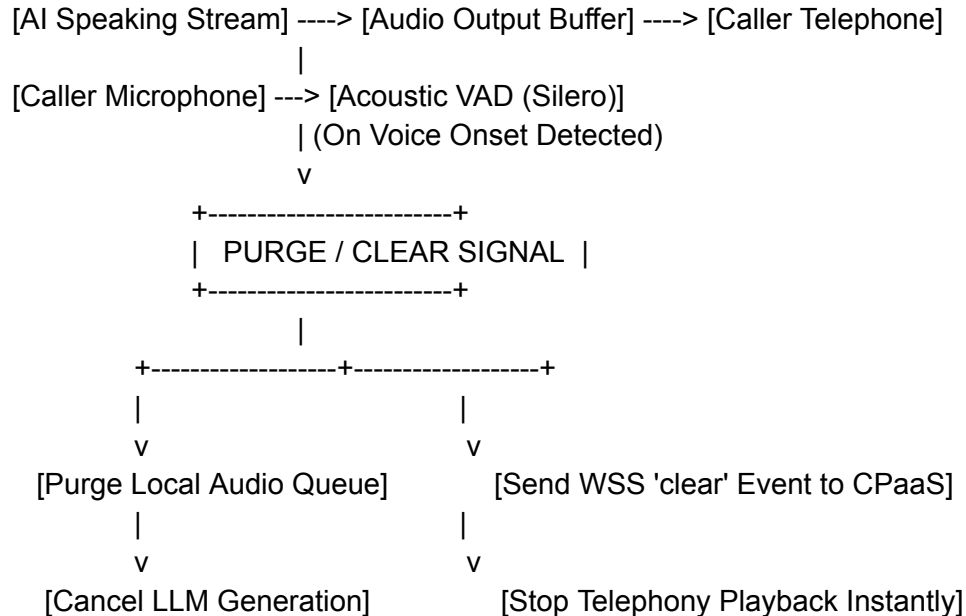
Latency Reduction Techniques

- Speculative LLM Execution: Start processing the LLM request when STT interim results reach high confidence ($>90\%$), even before final VAD endpointing fires. If the user continues speaking, cancel the speculative request.
-
- Early First-Sentence TTS Dispatch: Do not wait for the LLM to complete its entire response. Split output tokens by clause punctuation (commas, periods) and send the first 3–5 words immediately to the TTS engine.
-
- Filler Audio Injections: If tool calling or state lookup adds $>400\text{ms}$ latency, stream short, contextual audio fillers ("Let me check that...", "Hmm, one moment...") to hold the conversational turn without leaving dead silence.
-

4. Interruption & Turn-Taking Architecture

Handling barge-in smoothly separates amateur voice implementations from enterprise-grade systems.

USER SPEAKS WHILE AI IS PRODUCING AUDIO



Interruption Scenarios & Mitigations

1. User Barges In mid-sentence:
2.
 - Acoustic VAD detects speech ($>30\text{ms}$ energy above threshold).
 -
 - Orchestrator sends an immediate clear event to the telephony provider (e.g., Plivo/Exotel WebSocket clear command).
 -
 - The audio queue is flushed, LLM output generation is aborted, and the STT engine prioritizes the new user speech input.
 -
3. False Interruptions (Background noise, coughing, brief "mm-hmm"):
4.
 - Mitigation: Implement a two-stage confirmation window. Soft interrupt: Pause audio playback for 150ms . If the user speech stops within 200ms

(a quick "okay" or cough), resume playback from the pause point. If speech continues, execute a hard clear.

5. Mid-Utterance Thinking Pauses ("I was absent because... uh... my fever..."):

6.

- Mitigation: Do not rely solely on static silence timers. Use dynamic endpointing: lengthen silence threshold (800ms) when trailing words end with conjunctions or hesitation tokens ("because", "and", "uh"), and shorten it (300ms) when sentence completion syntax is detected.

5. Voice Quality & Speech Synthesis (TTS) Strategy

Provider / Model	Latency (TTFB)	Voice Naturalness	Streaming Support	Indian English Performance	Telugu Quality	Cost Profile
Cartesia Sonic	$\approx 90\text{ms} - 120\text{ms}$	High	WebSocket / Chunked	Excellent	Limited / Beta	$\$0.030 - \$0.060 / 1\text{k chars}$
ElevenLabs Turbo v2.5	$\approx 150\text{ms} - 250\text{ms}$	Industry Leading	WebSocket / Chunked	Excellent	Moderate	$\$0.150 - \$0.300 / 1\text{k chars}$
Deepgram Aura	$\approx 100\text{ms} - 140\text{ms}$	Moderate / Good	WebSocket	Good	Not Supported	$\$0.015 / 1\text{k chars}$
Sarvam AI (Bulbul v2/v3)	$\approx 200\text{ms} - 350\text{ms}$	High (Indic Native)	HTTP / WebSocket	Good	Best-in-Class (Native Pronunciation)	$\approx ₹15 - ₹30 / 10\text{k chars}$ (~ $\$0.018 - \0.036)

OpenAI Realtime Voice	$\approx 250\text{ms} - 400\text{ms}$	High	Native WebSockets	Good	Moderate	$\$0.060 - \$0.240 / \text{min}$ (Speech-to-Speech)
-----------------------	---------------------------------------	------	-------------------	------	----------	---

Technical Verdict on TTS

- For English Phase: Cartesia Sonic or ElevenLabs Turbo v2.5 provide the optimal blend of sub-150ms Time-To-First-Audio-Byte (TTFB) and fluid conversational prosody.
-
- For Telugu Phase: Sarvam AI (Bulbul v3) is necessary. Global models produce unnatural pitch contours and mispronounce regional phonemes, names, and places in Telugu.
-

6. Speech Recognition (STT) Strategy

Telephonic audio presents severe STT challenges: it is narrowband (sampled at 8kHz), compressed via lossy codecs (G.711 μ -law/A-law), and subjected to line noise and varying speaker volume.

Speech-to-Text Benchmark Comparison

STT Engine	Telephonic (8kHz) Accuracy	Indian Accent Handling	Code-Switching (Tanglish/Telugu-Eng)	Streaming Latency	Cost / Hour
Deepgram Nova-2 / Nova-3	High	High	Moderate (Trained on Indian audio)	$<150\text{ms}$	$\approx \$0.25 - \0.43
Sarvam STT	High	Native	Best-in-Class	$\approx 200\text{ms} - 300\text{ms}$	₹30 (\$ $\approx \$0.36$)
OpenAI Whisper (Cloud / Local Flash)	Moderate (Needs upsampling)	Moderate	Moderate	$>400\text{ms}$ (Batch/Chunked)	$\$0.36$

AssemblyA I Conformer- 2	High	High	Low	$\approx 250\text{ms}$	$\$0.37$
-----------------------------------	------	------	-----	------------------------	----------

Telephonic Audio Noise Strategy

1. Audio Resampling & Conditioning: Upsample incoming 8kHz PCM to 16kHz before processing if the target STT requires 16kHz input. Apply a high-pass filter ($<80\text{Hz}$) to eliminate telephonic DC offset and line hum.
- 2.
3. Vocabulary Prompting / Keyterm Injection: Inject contextual domain keywords (e.g., student names, branch names like "Electronics", reason categories like "Fever", "Casual Leave") into the STT engine configuration to force accurate recognition of proper nouns.
- 4.

7. LLM Reasoning & Dialogue Strategy

End-to-End Speech-to-Speech (S2S) vs. Cascaded Pipeline

CASCADED PIPELINE (Recommended for Enterprise/Deterministic Control):

[Audio In] -> [STT] -> [Text Prompt + Guardrails] -> [LLM] -> [JSON/Tool Call State] -> [TTS] -> [Audio Out]

END-TO-END SPEECH-TO-SPEECH (e.g., OpenAI Realtime / Gemini Live):

[Audio In] -----> [Multimodal Model] -----> [Audio Out]

Strategic Evaluation

- Speech-to-Speech (S2S) Models: Excellent natural expressiveness and low latency, but suffer from high costs ($\approx \$0.06 - \0.30min), hallucination risks, difficulty enforcing strict business logic (e.g., extracting precise student IDs), and limited support for direct Indian language telephony integration.
-
- Cascaded Hybrid Architecture (Recommended):
- - Orchestrator: High-speed deterministic state machine (Node.js/Python).
 -
 - LLM: Ultra-fast, instruction-tuned reasoning model (Groq Llama-3.1-8B, Gemini 2.0 Flash, or Claude 3.5 Haiku).
 -

- Function Calling / Tool Execution: Structured output extraction (JSON) runs concurrently to extract intent ("Reason: Medical", "Return Date: 2026-08-16").
-

8. Unit Economics & Realistic Cost Model

Per-Minute Operating Cost Breakdown (Production Enterprise Stack)

Cost Center	Vendor / Solution	Per-Minute Cost (USD)	Per-Minute Cost (INR)
Telephony Transit & WSS	Plivo / Exotel (India Outbound + Streaming)	$\$0.011$	₹0.94
Streaming STT	Deepgram Nova-2 / Sarvam STT	$\$0.005 - \0.007	₹0.42 - ₹0.60
LLM Reasoning	Gemini 2.0 Flash / Groq Llama-3.1-8B ($\approx 1k$ tokens/min)	$\$0.002 - \0.004	₹0.17 - ₹0.34
Streaming TTS	Cartesia / Sarvam Bulbul v3 (≈ 600 chars/min)	$\$0.003 - \0.018	₹0.25 - ₹1.50
Server Infra & Orchestration	AWS / Hetzner GPU/CPU Clusters	$\$0.002$	₹0.17
Total Estimated Cost / Min	Hybrid Enterprise AI Stack	$\mathbf{\$0.023 - \$0.042}$	$\mathbf{₹1.95 - ₹3.55}$ / minute

Cost Comparison across Scaling Tiers

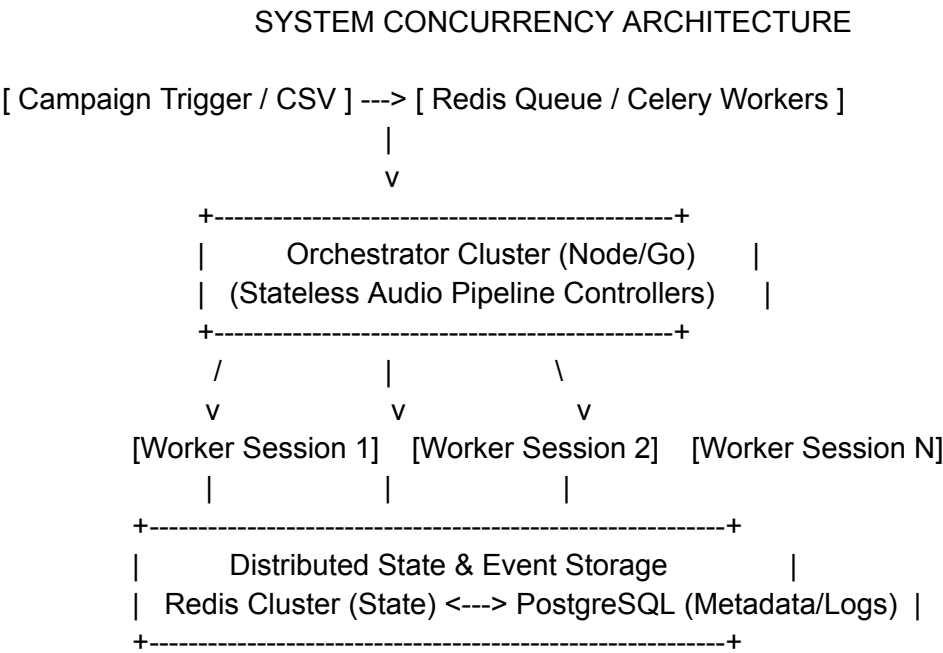
Human BPO Comparison in India: A human tele-caller in India costs approximately ₹4.00 to ₹8.00 per active conversational minute (factoring in salaries, office overhead, dialer software,

and idle time). AI calling operates at ₹2.00 to ₹3.50 per minute, delivering 50%–70% cost savings alongside instantaneous horizontal concurrency.

Campaign Volume	Total Estimated Minutes	AI Calling Platform Cost (INR)	Human Tele-Caller Cost (Est. ₹6/min)
100 Calls (Avg 1.5 min/call)	150 Mins	₹375 - ₹532	₹900
1,000 Calls	1,500 Mins	₹3,750 - ₹5,325	₹9,000
10,000 Calls	15,000 Mins	₹37,500 - ₹53,250	₹90,000
100,000 Calls	150,000 Mins	₹3,75,000 - ₹5,32,500	₹9,00,000

9. System Scalability & Throughput Architecture

Scaling to 1,000 concurrent active voice sessions requires a stateful, horizontally scalable architecture:



Architectural Bottlenecks & Solutions by Scaling Tier

- 1 to 10 Concurrent Calls:
 - - Single application server (Node.js/Python) maintaining WebSockets.
 -
 - Direct connections to third-party STT/TTS cloud APIs.
 -
- 10 to 100 Concurrent Calls:
 - - Bottleneck: Node.js event-loop congestion caused by audio transcoding/buffering.
 -
 - Solution: Decouple audio streaming handlers from application logic. Use a lightweight state machine controller and manage active call states in Redis.
 -
- 100 to 1,000 Concurrent Calls:
 - - Bottleneck: Third-party API rate limits (STT/TTS concurrency caps), telephony channel caps, and database write lock contention from stream events.
 -
 - Solution:
 - 1. Negotiate explicit rate-limit increases (Concurrency limits $>1,000$) with STT/TTS providers or deploy self-hosted STT/TTS containers on GPU clusters (e.g., vLLM + Triton Server).
 - 2.
 - 3. Utilize SIP Trunking with dynamic elastic channels rather than basic cloud REST APIs.
 - 4.
 - 5. Asynchronously push call transcripts and structured JSON results to a message queue (Kafka/RabbitMQ) for batch insertion into PostgreSQL.
 - 6.
 -

10. Regulatory & Compliance Framework in India

Navigating telecom compliance is critical for sustaining enterprise operations in India.

TRAI / DLT COMPLIANCE FLOWCHART

[Enterprise Call Intent]

```

|
+---> Is it Promotional / Sales?
|
|   +---> YES: Must use 140-Series Number Prefix
|       Scrub list against NCPR (Do-Not-Call) Database
|
+---> Is it Transactional / Service (e.g., Attendance, Banking)?
|
|   +---> YES: Must use 160-Series Number Prefix
|       Register Entity & Voice Script Templates on DLT Portal
|       Explicit Prior Consent required on file

```

1. Digital Personal Data Protection Act (DPDPA 2023):

2.

- Requires clear notice and consent mechanisms prior to processing personal data (student records, phone numbers).
-
- Call recordings and transcripts containing PII must be encrypted at rest (AES-256) and stored in in-region data centers within India.
-

3. Call Recording Consent:

4.

- Automated calls must begin with a clear, concise disclosure (e.g., "This is an automated AI system calling on behalf of ABC College. This call is recorded for record-keeping.").
-

5. Automated Calling Time Windows:

6.

- TRAI mandates commercial/promotional voice calls occur strictly between 09:00 AM and 09:00 PM IST. Transactional emergency calls are exempt but require compliant DLT template tagging.
-

11. Architectural Shift: English $\rightarrow$ Telugu Transition

Adapting an AI voice system to Telugu involves more than simple translation. It requires adjusting the underlying acoustic, linguistic, and state management models.

ENGLISH PIPELINE vs. TELUGU / TANGGLISH PIPELINE

ENGLISH PIPELINE:

[8kHz Audio] -> [Deepgram Nova-2] -> [Llama-3-8B] -> [Cartesia TTS] -> [Audio Out]

TELUGU / TANGGLISH PIPELINE:

[8kHz Audio] -> [Sarvam STT / Indic VAD] -> [Language ID / Routing Layer]

|
v

[Audio Out] <- [Sarvam Bulbul v3 TTS] <- [Llama-3 + Telugu System Prompt]

Key Technical Differences

1. Acoustic & VAD Adjustments:

2.

- Telugu conversational cadence features distinct vocal lengthenings and pitch inflection changes at sentence terminations (e.g., "Vasthunnara?" vs "Vasthunnaru").
-
- Standard VAD models tuned for English often misidentify vowel extensions in Telugu as silence or continued speech. VAD silence thresholds must be extended from 300ms to $450\text{ms} - 500\text{ms}$.
-

3. Handling Code-Switching (Tanglish / Telugu + English):

4.

- Native speakers frequently mix English words into Telugu conversation ("Nenu college ki fever valla raaleka poyanu").
-
- Generic STT models convert English words within Indic speech into phonetic non-words. You must employ a code-switching model (e.g., Sarvam STT) configured for mixed-script normalization.
-

5. Phonetic Entity Extraction (Names, Numbers, Dates):

6.

- LLM tokenizers often split Telugu script into numerous sub-word tokens, increasing token volume and inference latency by up to $3\times$.
-
- Mitigation: System prompts should enforce ASCII/English output for structured data extraction (e.g., output dates as 2026-08-14 and reasons in normalized English categories) while streaming Telugu text responses to the TTS engine.
-

12. Competitive Landscape Analysis

Provider / Platform	Core Differentiators	Platform Bottlenecks & Weaknesses	Indian Telephony Integration
Retell AI	Excellent low-latency orchestration, built-in turn-taking, developer-friendly Webhooks.	High markup on underlying STT/TTS; restricted customization for low-level audio buffering.	Good (Supports custom SIP Trunking / Plivo).
Vapi	Flexible, modular architecture; allows easy swapping of STT/LLM/TTS models.	Higher latency variations due to multiple API orchestrations; requires careful tuning.	High (Native SIP / WebSockets via Plivo & Twilio).
Bland AI	Native custom telephony infrastructure, built for high enterprise outbound volume.	Proprietary "black-box" model execution; limited control over exact VAD and turn-taking logic.	Moderate (Primarily optimized for US/EU PSTN).
Bolna.ai	Open-source platform built specifically for Indian voice AI workflows; native support for Indic TTS/STT.	Infrastructure requires active engineering maintenance at large scale.	Native (Pre-built integrations with Indian DLT, Vobiz, Exotel).

13. Critical Risks & Failure Modes Matrix

Risk Factor	Root Cause	Impact	Mitigation Strategy
Robotic Latency Spikes	Sequential API processing, LLM cold starts, network jitter	High (Caller hangs up)	Implement speculative execution, streaming chunked TTS, and host orchestration in Mumbai (ap-south-1).

Barge-In Echo Loops	Telephony speaker output bleeds into microphone input	High (AI interrupts itself)	Enable strict echo cancellation (AEC) at the telephony level and enforce a 150ms VAD grace period post-playback.
Hallucinated State Tracking	Unstructured LLM conversation drift	Medium (Corrupts reporting data)	Separate conversation generation from state tracking. Use deterministic state machines to update database values via structured tool calls.
DLT / Number Blocking	Calling non-consented numbers via incorrect number series	Fatal (Telecom blocking)	Secure 160-series numbers for transactional workflows, register principal entities on DLT, and maintain explicit opt-in records.
Poor Telugu Name Extraction	STT phonetically misinterpreting regional student names	Medium (Bad data export)	Pre-load CSV student lists into the LLM context prompt so entity recognition matches against known candidate names.

14. Recommended Master Technology Strategy

Recommended Technology Stack

+-----+			
PRODUCTION STACK			
[Telephony Layer]	Plivo / Exotel SIP Trunking (160 Series Numbers)		
[Transport Protocol]	Bidirectional WebSockets (PCM 8kHz / Slinc)		
[Framework Core]	Pipecat / LiveKit Agents Engine (Python/Go)		
[VAD Engine]	Silero VAD (Sub-10ms Acoustic Detection)		
+-----+			

[English Stack]	[Telugu / Multilingual Stack]	
STT: Deepgram Nova-2	STT: Sarvam STT (Code-Switching Enabled)	
LLM: Groq Llama-3.1-8B	LLM: Gemini 2.0 Flash / Claude 3.5 Haiku	
TTS: Cartesia Sonic	TTS: Sarvam Bulbul v3	
+-----+		
[Application Backend]	FastAPI / Node.js Microservices	
[Cache & Session State]	Redis Cluster	
[Database & Analytics]	PostgreSQL (JSONB Transcript & Call Result Storage)	
[Infrastructure Host]	AWS India (ap-south-1 Mumbai)	
+-----+		

Implementation Roadmap

PHASE 1: Core Architecture & English Prototype

- 1. Deploy Pipecat/LiveKit Orchestrator on AWS Mumbai (ap-south-1)
- 2. Provision Plivo/Exotel Indian Test Numbers with WebSockets
- 3. Wire Pipeline: Silero VAD -> Deepgram STT -> Groq LLM -> Cartesia TTS
- 4. Build CSV Ingestion, Campaign Scheduler, and Structured Data Exporter

PHASE 2: Interruption Optimization & State Reliability

- 1. Implement Dual-VAD and WebSocket frame purge signals
- 2. Add Speculative Execution to reduce perceived turn latency to <700ms
- 3. Implement Deterministic State Machine for Call Disposition (Answered, Busy, Reason)

PHASE 3: Telugu & Tanglish (Code-Switching) Integration

- 1. Integrate Sarvam STT and Bulbul v3 TTS into the Pipeline
- 2. Calibrate VAD Silence Windows (450ms) for Telugu Vocal Cadence
- 3. Train Prompt Layer for Entity Normalization (Converting spoken Telugu names/dates to English JSON)

PHASE 4: Enterprise Scale & Telecom Compliance

- 1. Obtain DLT Registration & Provision 160-Series Number Blocks
- 2. Scale Architecture with Redis Session Cache and Elastic SIP Trunking
- 3. Perform Load Testing up to 1,000 Concurrent Active Calls

Architectural Synthesis

To realize your vision of an ultra-natural AI voice platform:

1. Avoid SIM Cards: Use Plivo or Exotel Cloud Telephony over WebSockets to ensure Indian compliance, full-duplex control, and scalability.
- 2.
3. Prioritize Latency: Choose high-performance components like Cartesia Sonic (English), Sarvam AI (Telugu), and Groq Llama-3.1 to keep total round-trip response time below 800ms .
- 4.
5. Control Interruption Logic: Build a full-duplex pipeline capable of instantly flushing stale audio buffers upon detecting user speech.
- 6.
7. Ensure Regulatory Compliance: Register your entity on the DLT portal and secure 160-series numbers for legitimate service and attendance calling.

qwen 3.8 max

As a senior voice-AI architect and telecom engineer, I have evaluated your project requirements against the current Indian telecommunications landscape, real-time AI inference capabilities, and TRAI (Telecom Regulatory Authority of India) regulations. Your goal of building a "highly natural" voice agent is technically feasible today, but the Indian market presents unique regulatory and infrastructural hurdles that invalidate several standard "Western" voice-AI assumptions. Below is a research-grade, independent analysis of your project.

1. Telephony: The Indian Reality

Challenge your assumption: Do not use SIM cards, eSIMs, or GSM Gateways (SIM farms) for this project.

Using GSM gateways to route automated bulk calls bypasses standard interconnect charges and is classified as illegal telecom fraud by TRAI

decoded.legal

. It will result in immediate blocking of your hardware and severe legal penalties

www.infobip.com

Best Options for India:

- Cloud Telephony (Aggregators): Providers like Exotel, Plivo, and Ozonetel are the standard for Indian Voice AI
- caller.digital
- . They provide DLT-compliant virtual numbers and handle the complex KYC required by Indian telecom operators (Jio, Airtel, Vi).
- Programmable Voice APIs: Plivo offers excellent WebSocket-based "Audio Streaming" specifically designed for AI agents
- plivo.com
- . EnableX is another strong local contender with real-time raw audio WebSocket streaming
- www.enablex.io
- .
- SIP Trunking: For high volume (10k+ calls/day), direct SIP trunks via Tata Tele or Jio Business offer the lowest per-minute rates (₹0.40–₹0.60/min) but require a Registered Telemarketer license and high upfront deposits.
- Pricing & Routing: Twilio is generally too expensive for domestic Indian outbound calls (₹1.20–₹1.50/min)
- exotel.com
- . Local providers like Plivo or Exotel charge between ₹0.50 and ₹0.90 per minute for outbound mobile calls
- caller.digital

- ,
- vobiz.ai
- .

2. Real-Time Voice Architecture

To achieve "human-like" quality, you must move away from standard request-response loops and adopt a Continuous Streaming Pipeline.

- Orchestration Frameworks: Do not build the pipeline from scratch. Use Pipecat (by Daily.co) or LiveKit Agents

- www.cekura.ai

- .
 - *Pipecat* uses a pipeline architecture (VAD → STT → LLM → TTS) giving you granular control over barge-in logic and audio buffering
 - thinnest.ai
 - .
 - *LiveKit* uses a "room" model based on WebRTC, which is superior if you plan to add multimodal features (video/avatar) later
 - www.cekura.ai
 - .

- Audio Transport: You must stream raw PCM/Mulaw audio via WebSockets from the telephony provider to your media server.
- State Management: Redis is required to maintain the independent state of thousands of concurrent calls, storing the conversation history and tool-call status for every active session.

3. Latency Targets & Pipeline

In human conversation, a gap of >1000ms feels "robotic." Your target for Time-To-First-Audio (TTFA) should be < 600ms.

The Latency Pipeline:

1. Endpointing (50ms): Detecting the user stopped speaking.
2. STT Streaming (200ms): Transcribing audio *while* the user is speaking.
3. LLM Inference (150ms): Generating the first token of the response.
4. TTS Generation (100ms): Synthesizing the first audio chunk.
5. Network/Playback (50ms): Sending audio back to the phone.

Strategy: You must use Streaming APIs for every component. Do not wait for the LLM to finish generating a full sentence before sending it to the TTS. Send chunks of 10-20 words as they are generated.

4. Interruption and Turn-Taking (Barge-In)

This is the hardest engineering challenge.

- Acoustic Echo Cancellation (AEC): When the AI speaks, the user's phone mic might pick up the AI's voice. If your system doesn't filter this out, it will "hear itself" and interrupt itself.
- Semantic Endpointing: Standard systems wait for 800ms of silence. High-end systems analyze the *transcript* to see if the user finished a thought. If the user pauses but the sentence is grammatically incomplete, the AI waits.
- Cancel-and-Clear: When the user interrupts, the system must instantly drop the TTS buffer, cancel the LLM generation, and pass the new audio to the STT. Frameworks like Pipecat handle this "barge-in" logic natively
- [thinnest.ai](#)
- .

5. Voice Quality (TTS)

For Indian English and eventual Telugu, naturalness is key.

- Cartesia (Sonic): Currently the latency leader (~40ms TTFA)
- [deepgram.com](#)
- . It is optimized for real-time conversational AI and includes "fillers" (ums, ahs) that make it sound human.
- ElevenLabs (Turbo v2.5 / Multilingual v2): The industry standard for emotional range and voice cloning. It supports Indian accents well and has a specific multilingual model that covers Telugu.
- Avoid: Standard cloud TTS (AWS Polly, Google WaveNet) as they lack the "breathiness" and pacing variations of human speech.

6. STT Quality for Telephony

- The Problem: Phone calls use 8kHz narrowband audio, which degrades quality.
- The Solution: Deepgram Nova-3 or AssemblyAI Universal-1. Deepgram is specifically optimized for telephony audio and Indian accents, offering <300ms latency
- [deepgram.com](#)
- ,
- [futureagi.substack.com](#)
- .

- Whisper Warning: While OpenAI's Whisper is powerful, the Large-v3 model often struggles with 8kHz telephony compression and heavy Indian accents compared to specialized commercial models
- www.hirefinn.ai
- ,
- novascribe.ai
- .

7. LLM Strategy

Challenge your assumption: You do *not* need a massive reasoning model for 90% of this project.

- The "College Absent" Use Case: This is a deterministic flow. You need a fast, low-latency model like Llama 3.1 70B (via Groq) or GPT-4o-mini. Groq offers inference speeds that are virtually instantaneous, which is critical for voice
- blog.portaone.com
- .
- Architecture: Use a "Router" model. A fast model handles the conversation flow and turn-taking. Only route to a heavy model (Claude 3.5 Sonnet / GPT-4o) if the user asks a complex, out-of-domain question that requires deep reasoning.

8. Cost Estimates (Per Minute)

Based on current API pricing

	softcery.com
,	
	inworld.ai
,	
	blog.portaone.com
:	

- Telephony (India): ~\$0.012/min (₹1.00)
- STT (Deepgram): ~\$0.007/min
- LLM (Groq/GPT-4o-mini): ~\$0.005/min
- TTS (Cartesia/ElevenLabs): ~\$0.030/min
- Total AI Cost: ~\$0.054 per minute (approx. ₹4.50/min).

Scale Comparison:

- 1,000 calls (3 min avg) = ~\$162
- 100,000 calls = ~\$16,200

- *Human Alternative:* A human BPO agent in India costs approx. ₹15–₹25 per minute (fully loaded). AI is roughly 5x-8x cheaper and infinitely more scalable.

9. Scalability Bottlenecks

- 1–10 Calls: Handled easily by a single Node.js/Python server.
- 100 Calls: Requires load balancing and managing WebSocket file descriptors.
- 1,000+ Calls: The bottleneck shifts to SIP Trunk Capacity and Media Server CPU. You will need a Kubernetes cluster (e.g., using LiveKit or Jitsi) to distribute the audio processing load across multiple nodes.

10. Indian Legal & Telecom (Critical)

This is where most international projects fail in India.

- DLT Registration: Under TRAI regulations, every commercial/transactional call must be registered on a Distributed Ledger Technology (DLT) platform
- thinnest.ai
- . You must register as a "Principal Entity" (the College) and approve your "Headers" (Caller ID names) and "Teasers" (Call purpose templates).
- Number Series: Promotional calls use the 140-series. Service/Transactional calls (like college attendance) use the 160-series or specific whitelisted headers
- www.linkedin.com
- .
- AI Disclosure: Recent TRAI updates mandate that AI voice agents must disclose they are artificial if asked, or at the start of the call
- qcall.ai
- .
- Penalties: Violating DND (Do Not Disturb) registries or DLT rules can attract penalties up to ₹50 Lakhs and disconnection of all telecom resources
- conversailabs.com
- .

11. The English → Telugu Transition

Moving to Telugu is not just a "translation" task; it is a linguistic engineering challenge.

- Code-Switching (Telugish): Indian users rarely speak pure Telugu. They say, *"Nenu bus miss ayyindi, so late ga vasthanu."* Standard TTS/STT models fail here.
- STT: You must switch to AI4Bharat's IndicConformer models, which are specifically trained on Indian code-mixed datasets and telephony audio

- models.ai4bharat.org
- ,
- ai4bharat.iitm.ac.in
- .
- TTS: AI4Bharat's IndicTTS and Indic-Parler-TTS offer the most natural Telugu prosody
- github.com
- ,
- huggingface.co
- . ElevenLabs Multilingual v2 is good but may struggle with specific regional slang unless you use phonetic prompting.

12. Competitive Landscape Analysis

- Bland AI / Air AI: Proprietary, end-to-end platforms. Good for US markets, but they lack the deep integration with Indian DLT compliance and local SIP routing required for high connection rates in India.
- Vapi / Retell AI: These are "orchestration layers." They allow you to plug in Twilio/Plivo and Deepgram. They are excellent for developer flexibility but require you to manage the Indian telecom compliance wrapper yourself.
- Differentiation: The winning systems in India will be those that master Semantic Endpointing and DLT Compliance, not just those with the best voices.

13. Critical Risks (The "Project Killers")

1. TRAI Blocking: If your DLT templates are not approved or you call DND numbers, your numbers will be blocked within 24 hours.
2. Robotic Turn-Taking: If you use a fixed 800ms silence timeout, the AI will constantly interrupt people who are just thinking. You *must* implement semantic endpointing.
3. Hallucinations: If the AI tells a student "You are expelled" because the LLM hallucinated, the liability falls on the college. You need strict "System Prompts" and "Tool Calling" constraints.
4. Audio Degradation: 8kHz telephony audio + background noise (traffic/fans in India) will destroy standard Whisper models.

14. Recommended Technology Strategy

Based on the research, here is the optimal stack for your prototype and MVP:

- Telephony: Plivo or Exotel. Use their WebSocket Audio Streaming APIs. Ensure your "Principal Entity" KYC is completed before writing code.
- Orchestration Framework: Pipecat (Python). It gives you the lowest-level control over the VAD and Barge-in logic, which is essential for "natural" conversation.
- STT: Deepgram Nova-3 (Telephony model, streaming).
- LLM: Groq (Llama 3.1 70B). The inference speed is unmatched and critical for keeping latency under 600ms.
- TTS: Cartesia Sonic (for lowest latency) or ElevenLabs Turbo v2.5 (for best naturalness).
- Backend: Python (FastAPI) + Redis (State) + PostgreSQL (Campaign Data).
- Deployment: Host your media servers in the AWS Mumbai (ap-south-1) region to minimize network latency to Indian telecom exchanges.

Final Advice: Do not start with Telugu. Spend 3 months perfecting the "Interruption/Barge-in" logic and the DLT compliance workflow in English. If the English version feels robotic, the Telugu version will be unlistenable due to the added complexity of code-mixing.